BAB VI

React *Progressive Web App* (PWA) II

6.1. Tujuan

1. Praktikan dapat memahami konsep dasar teknologi *Progressive Web App*
2. Praktikan mampu melakukan proses *deployment project* PWA dengan benar menggunakan layanan *hosting* Vercel
3. Praktikan dapat memahami dan mengimplementasikan Progressive Web App (PWA) menggunakan ReactJS
4. Praktikan dapat memahami konsep, menggunakan, dan mengintegrasikan RESTful API untuk *project* PWA
5. Praktikan dapat membangun Single Page Application (SPA) dengan React Hooks, error handling, state management, dan implementasi localStorage untuk data persistence.

6.2. Alat dan Bahan

1. Laptop
2. *Smartphone*
3. Browser
4. Text Editor (disarankan menggunakan Visual Studio Code)
5. Node.js
6. Git
7. Akun Github aktif untuk daftar Vercel

6.3. Dasar Teori

6.3.1. Progressive Web App (PWA)

Progressive Web App (PWA) adalah aplikasi web yang menggunakan pendekatan *progressive enhancement* untuk memberikan pengalaman seperti aplikasi *native*. PWA dapat diinstal pada perangkat pengguna, bekerja secara *offline* melalui *service worker*, dan menggunakan web *app manifest* untuk mengontrol perilaku instalasi serta tampilan. Tujuan utama PWA adalah menghadirkan aplikasi web yang andal, cepat, dan menarik, namun tetap berbasis teknologi web standar seperti HTML, CSS, dan JavaScript.

(Sumber: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps)

6.3.2. React JS

React JS adalah pustaka JavaScript untuk membangun antarmuka pengguna (*user interface*). React menggunakan pendekatan *component-based* sehingga setiap bagian tampilan dipecah menjadi komponen yang dapat digunakan kembali. React juga menerapkan konsep *declarative programming*, di mana pengembang hanya perlu mendefinisikan bagaimana tampilan seharusnya pada suatu kondisi, dan React akan mengelola perubahan data secara otomatis melalui *Virtual* DOM. Tujuan utama React adalah memudahkan pengembang membuat antarmuka dinamis dan interaktif dengan struktur kode yang modular dan mudah dikelola.

(Sumber: https://react.dev)

6.3.3. Vite

Vite (diucapkan /vit/, dari bahasa Prancis yang berarti “cepat”) adalah sebuah *build tool* modern yang berfungsi untuk menjalankan dan membangun proyek aplikasi web *front-end*. Tujuan utama Vite adalah memberikan pengalaman pengembangan yang jauh lebih cepat dan efisien dibandingkan dengan alat-alat generasi sebelumnya. Vite memanfaatkan *native* ES modules pada browser modern untuk mempercepat waktu startup saat pengembangan, serta menggunakan *Rollup* untuk proses *build* produksi yang optimal.

(Sumber: https://vite.dev/guide/)

6.3.4. API

Application Programming Interface (API) adalah sekumpulan aturan dan protokol yang memungkinkan aplikasi atau sistem perangkat lunak berkomunikasi dan bertukar data. API mendefinisikan cara komponen perangkat lunak saling berinteraksi melalui permintaan (*request*) dan respons (*response*) yang terstruktur. API banyak digunakan untuk menghubungkan *frontend* dan *backend*, misalnya melalui layanan RESTful atau GraphQL, sehingga data dapat dikirim dan diterima secara efisien antar sistem.

(Sumber: https://www.ibm.com/think/topics/api)

6.3.5. Axios

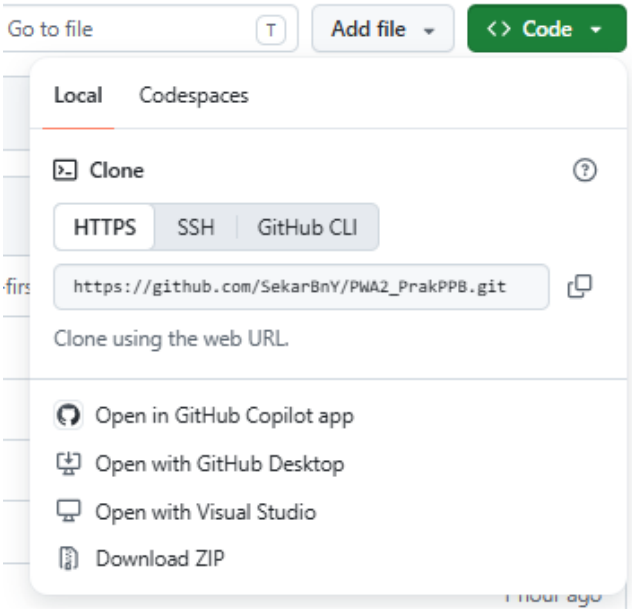
Axios adalah pustaka JavaScript berbasis *Promise* yang digunakan untuk melakukan permintaan HTTP dari browser maupun Node.js. Axios mendukung berbagai metode seperti GET, POST, PUT, dan DELETE, serta menyediakan fitur *interceptors*, *request cancellation*, dan transformasi data otomatis (misalnya format JSON). Keunggulan utama Axios adalah kemudahan integrasi dengan *frontend framework* seperti React, kemampuan penanganan kesalahan (*error handling*) yang baik, serta dukungan penuh untuk operasi asinkron menggunakan *async* dan *await*.

(Sumber: https://axios-http.com/docs/intro)

6.4. Langkah Kerja

6.4.1. *Clone Repository* Github Modul 5

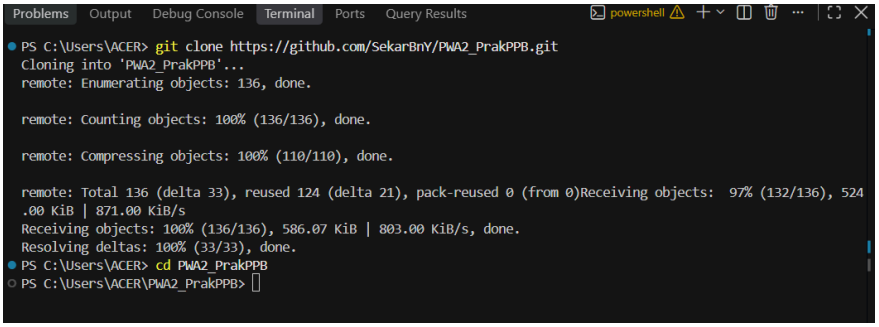
1. Buka *link* berikut https://github.com/SekarBnY/PWA2_PrakPPB.git
2. Klik tombol *Code* kemudian *copy link* yang tampil



Gambar 6. 1 *Clone Repository* Github Modul 4

3. Buka *terminal* dan ikuti *command* di bawah ini untuk *clone* dan membuka *project* pada IDE

```
>git clone https://github.com/SekarBnY/PWA2_PrakPPB.git
>cd PWA2_PrakPPB
```



Gambar 6. 2 Membuka *project* hasil *clone* pada *local*

6.4.2. Integrasi *Frontend* PWA dengan API

1. Buat *file* konfigurasi API pada src/config/api.js sebagai berikut

```
import axios from 'axios'
```

```
//
------------------------------------------------------------
-----------------
// Base URL & Supabase Key Configuration
// Read from environment variables (.env)
//
------------------------------------------------------------
-----------------
const BASE_URL = import.meta.env.VITE_API_BASE_URL || ''
const SUPABASE_KEY =
  import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY ||
  import.meta.env.VITE_SUPABASE_ANON_KEY ||
  ''

const IS_SUPABASE =
                BASE_URL.includes('supabase.co')        ||
Boolean(SUPABASE_KEY)

if (!BASE_URL) {
  console.warn(
     '[api] VITE_API_BASE_URL is not set. API requests will
use relative paths.',
  )
}

//
------------------------------------------------------------
-----------------
// Axios instance
//
------------------------------------------------------------
-----------------
const apiClient = axios.create({
  baseURL: BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
  timeout: 10000, // 10 seconds
})

//
------------------------------------------------------------
-----------------
// Request interceptor — attach Supabase headers (apikey &
Bearer auth)
//
------------------------------------------------------------
-----------------
apiClient.interceptors.request.use(
  (config) => {
    if (IS_SUPABASE && SUPABASE_KEY) {
      config.headers['apikey'] = SUPABASE_KEY
      if (!config.headers['Authorization']) {
                config.headers['Authorization']  =  `Bearer
${SUPABASE_KEY}`
```

```
      }
    }
    return config
  },
  (error) => Promise.reject(error),
)

//
------------------------------------------------------------
-----------------
// Response interceptor — unwrap response and normalise
errors
//
------------------------------------------------------------
-----------------
apiClient.interceptors.response.use(
  (response) => response.data,
  (error) => {
    let message = 'An unexpected error occurred'
    let status = null

    if (error.response) {
      status = error.response.status
      message =
        error.response.data?.message ||
        error.response.data?.msg ||
        error.response.data?.error ||
        error.response.data?.error_description ||
        `Server error (${status})`
    } else if (error.request) {
      message =
         'Unable to reach the server. Check your connection
and try again.'
    } else {
      message = error.message || message
    }

    return Promise.reject({
      message,
      status,
      code: error.response?.data?.code,
      raw: error.response?.data,
    })
  },
)

export { apiClient, BASE_URL, SUPABASE_KEY, IS_SUPABASE }
```

2. Buat *environment file* (.env) pada *root project* sebagai berikut

```
VITE_API_BASE_URL=https://sxunubxupdxyupnebuid.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_TXg9-r3ZVI8YZ
MzE90nwsQ_PU9GAmzg
```

3. Buat *custom hooks* (hooks/usexxx.js) untuk menggunakan *services*

src\hooks\useGuns.js

```
import { useState, useEffect, useCallback } from 'react'
import gunService from '../services/gunService'

//
------------------------------------------------------------
-----------------
// useGuns — fetch the paginated / filtered product list
from the API.
//
// Returns { guns, loading, error, pagination, refetch }.
// When the API is unreachable the `error` string will be
set and `guns`
// will remain an empty array so the UI can show a proper
error state.
//
------------------------------------------------------------
-----------------

export function useGuns(params = {}) {
  const [guns, setGuns] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [pagination, setPagination] = useState(null)

   // Serialise params so the effect re-runs when they
change.
  const paramKey = JSON.stringify(params)

  const fetchGuns = useCallback(async () => {
    try {
      setLoading(true)
      setError(null)

      const response = await gunService.getGuns(params)

      if (response.success) {
        setGuns(response.data || [])
        setPagination(response.pagination || null)
      } else {
           setError(response.message || 'Failed to fetch
products.')
      }
    } catch (err) {
        setError(err.message || 'An error occurred while
fetching products.')
      setGuns([])
    } finally {
      setLoading(false)
    }
                          //      eslint-disable-next-line
react-hooks/exhaustive-deps
```

```
  }, [paramKey])

  useEffect(() => {
    fetchGuns()
  }, [fetchGuns])

    return { guns, loading, error, pagination, refetch:
fetchGuns }
}

//
------------------------------------------------------------
-----------------
// useGunById — fetch a single product by its UUID.
//
// Returns { gun, loading, error, refetch }.
//
------------------------------------------------------------
-----------------

export function useGunById(id) {
  const [gun, setGun] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  const fetchGun = useCallback(async () => {
    if (!id) {
      setLoading(false)
      return
    }
    try {
      setLoading(true)
      setError(null)

      const response = await gunService.getGunById(id)

      if (response.success) {
        setGun(response.data)
      } else {
        setError(response.message || 'Product not found.')
      }
    } catch (err) {
         setError(err.message || 'Failed to load product
details.')
      setGun(null)
    } finally {
      setLoading(false)
    }
  }, [id])

  useEffect(() => {
    fetchGun()
  }, [fetchGun])

  return { gun, loading, error, refetch: fetchGun }
}
```

src\hooks\useReviews.js

```
import { useState, useEffect, useCallback } from 'react'
import reviewService from '../services/reviewService'

//
------------------------------------------------------------
-----------------
// useReviews — fetch reviews for a specific product.
//
------------------------------------------------------------
-----------------

export function useReviews(gunId) {
  const [reviews, setReviews] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  const fetchReviews = useCallback(async () => {
    if (!gunId) {
      setLoading(false)
      return
    }
    try {
      setLoading(true)
      setError(null)

                              const    response    =    await
reviewService.getReviews(gunId)

      if (response.success) {
        setReviews(response.data || [])
      } else {
            setError(response.message  ||  'Failed  to  fetch
reviews.')
      }
    } catch (err) {
        setError(err.message  ||  'An  error  occurred  while
fetching reviews.')
      setReviews([])
    } finally {
      setLoading(false)
    }
  }, [gunId])

  useEffect(() => {
    fetchReviews()
  }, [fetchReviews])

   return { reviews, loading, error, refetch: fetchReviews
}
}
```

```
//
------------------------------------------------------------
-----------------
// useCreateReview — submit a new review for a product.
//
------------------------------------------------------------
-----------------

export function useCreateReview() {
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  const [success, setSuccess] = useState(false)

  const createReview = async (gunId, reviewData) => {
    try {
      setLoading(true)
      setError(null)
      setSuccess(false)

                            const    response    =    await
reviewService.createReview(gunId, reviewData)

      if (response.success) {
        setSuccess(true)
        return response.data
      } else {
           setError(response.message || 'Failed to submit
review.')
        return null
      }
    } catch (err) {
         setError(err.message || 'An error occurred while
submitting review.')
      return null
    } finally {
      setLoading(false)
    }
  }

  return { createReview, loading, error, success }
}
```

src\hooks\useFavorites.js

```
import { useState, useEffect, useCallback } from 'react'
import favoriteService from '../services/favoriteService'
import       {       getUserIdentifier       }       from
'../services/userService'

//
------------------------------------------------------------
-----------------
//  localStorage  fallback  —  used  when  the  backend  is
unavailable so that
```

```
// favourites are still fully functional offline.
//
------------------------------------------------------------
-----------------

const LOCAL_KEY = 'bore_barrel_favorites'

const readLocalFavs = () => {
  try {
    const raw = localStorage.getItem(LOCAL_KEY)
    return raw ? JSON.parse(raw) : []
  } catch {
    return []
  }
}

const writeLocalFavs = (ids) => {
  try {
    localStorage.setItem(LOCAL_KEY, JSON.stringify(ids))
  } catch {
    /* quota exceeded — silently fail */
  }
}

//
------------------------------------------------------------
-----------------
// useFavorites — list all favourited product IDs.
//
//   First   attempts   the   server;   on   failure   uses
localStorage.
//
------------------------------------------------------------
-----------------

export function useFavorites() {
  const [favorites, setFavorites] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [isLocal, setIsLocal] = useState(false)
  const userIdentifier = getUserIdentifier()

  const fetchFavorites = useCallback(async () => {
    try {
      setLoading(true)
      setError(null)

                            const    response    =    await
favoriteService.getFavorites(userIdentifier)

      if (response.success) {
        setFavorites(response.data || [])
        setIsLocal(false)
      } else {
        throw new Error(response.message)
      }
```

```
    } catch {
      // Fallback to localStorage
      setFavorites(readLocalFavs())
      setIsLocal(true)
    } finally {
      setLoading(false)
    }
  }, [userIdentifier])

  useEffect(() => {
    fetchFavorites()
  }, [fetchFavorites])

   return { favorites, loading, error, isLocal, refetch:
fetchFavorites }
}

//
------------------------------------------------------------
-----------------
//  useToggleFavorite  —  toggle  a  product  in/out  of
favourites.
//
------------------------------------------------------------
-----------------

export function useToggleFavorite() {
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  const userIdentifier = getUserIdentifier()

  const toggleFavorite = async (gunId) => {
    try {
      setLoading(true)
      setError(null)

                            const    response    =    await
favoriteService.toggleFavorite({
        recipe_id: gunId,
        user_identifier: userIdentifier,
      })

      if (response.success) {
        return response.data
      }
      throw new Error(response.message)
    } catch {
      // Fallback: toggle in localStorage
      const current = readLocalFavs()
       const idx = current.findIndex((id) => String(id) ===
String(gunId))
      let isFavorited

      if (idx === -1) {
        current.push(gunId)
        isFavorited = true
```

```
      } else {
        current.splice(idx, 1)
        isFavorited = false
      }
      writeLocalFavs(current)
      return { is_favorited: isFavorited }
    } finally {
      setLoading(false)
    }
  }

  return { toggleFavorite, loading, error }
}

//
------------------------------------------------------------
-----------------
// useIsFavorited — convenience wrapper for a single
product.
//
------------------------------------------------------------
-----------------

export function useIsFavorited(gunId) {
   const { favorites, loading: fetchLoading, refetch } =
useFavorites()
  const { toggleFavorite: toggle, loading: toggleLoading }
= useToggleFavorite()

   // Check both server-format (objects with .id) and
local-format (plain ids)
  const isFavorited = favorites.some((fav) => {
    const favId = typeof fav === 'object' && fav !== null
? (fav.id ?? fav.product_id ?? fav.recipe_id) : fav
    return String(favId) === String(gunId)
  })

  const toggleFavorite = async () => {
    const result = await toggle(gunId)
    if (result) await refetch()
    return result
  }

  return {
    isFavorited,
    loading: fetchLoading || toggleLoading,
    toggleFavorite,
  }
}

export { getUserIdentifier }
```

4. Implementasi *services* (src/services/xxxService.js) untuk membuat *method fetch* API sesuai dengan *endpoint*

src/services/favoriteService.js

```
import { apiClient, IS_SUPABASE } from '../config/api'
import { normalizeProduct } from './gunService'

//
---------------------------------------------------------------
-----------------
// FavoriteService — server-side favourites.
// Supports both Supabase (public.favorites with products
foreign key join)
// and university API (/api/v1/favorites).
//
// NOTE: When server/RLS is offline or restricted,
useFavorites hook falls back
// to localStorage automatically so favorites always work.
//
---------------------------------------------------------------
-----------------

class FavoriteService {
  /**
   * Retrieve all favourited products for the given user.
   */
  async getFavorites(userIdentifier) {
    if (IS_SUPABASE) {
      const data = await apiClient.get(

`/rest/v1/favorites?user_id=eq.${userIdentifier}&select=pr
oduct_id,created_at,products(*)`,
      )
      const list = Array.isArray(data)
        ? data.map((fav) => {
            const prod = fav.products ?
normalizeProduct(fav.products) : null
            return {
              ...(prod || {}),
              id: fav.product_id,
              recipe_id: fav.product_id,
              product_id: fav.product_id,
              created_at: fav.created_at,
            }
          })
        : []

      return { success: true, data: list }
    }

    return apiClient.get('/api/v1/favorites', {
      params: { user_identifier: userIdentifier },
    })
  }
```

```
  /**
   * Toggle a product's favourite state (add ↔ remove).
   * @param {Object} data  { recipe_id, user_identifier }
   */
  async toggleFavorite(data) {
    if (IS_SUPABASE) {
      const targetId = Number(data.recipe_id ||
data.gun_id)
      const userId = data.user_identifier

      // Check if already favorited
      const existing = await apiClient.get(
        `/rest/v1/favorites?product_id=eq.
targetId&user_id=eq.${userId}&select=*`,
      )

      if (Array.isArray(existing) && existing.length > 0)
{
        // Remove from favorites
        await apiClient.delete(
          `/rest/v1/favorites?product_id=eq.
targetId&user_id=eq.${userId}`,
        )
        return { success: true, data: { is_favorited:
false } }
      } else {
        // Add to favorites
        await apiClient.post(
          '/rest/v1/favorites',
          { product_id: targetId, user_id: userId },
          { headers: { Prefer: 'return=minimal' } },
        )
        return { success: true, data: { is_favorited: true
} }
      }
    }

    return apiClient.post('/api/v1/favorites/toggle',
data)
  }
}

export default new FavoriteService()
```

src/services/gunService.js

```
import { apiClient, IS_SUPABASE } from '../config/api'

//
------------------------------------------------------------
```

```
-----------------
// Normalizer — standardizes product shape whether
received from Supabase
// (public.products) or from the university API
(/api/v1/recipes).
//
-----------------------------------------------------------
-----------------
export const normalizeProduct = (item) => {
  if (!item) return null

  // Resolve specifications from Supabase `specs` (text
array) or API `ingredients`
  let specsList = []
  if (Array.isArray(item.specs)) {
    specsList = item.specs.map((s, idx) => {
      if (typeof s === 'string') {
        const colonIndex = s.indexOf(':')
        if (colonIndex > -1) {
          return {
            id: `spec-${idx}`,
            name: s.substring(0, colonIndex).trim(),
            quantity: s.substring(colonIndex + 1).trim(),
          }
        }
        return {
          id: `spec-${idx}`,
          name: 'Specification',
          quantity: s.trim(),
        }
      }
      return s
    })
  } else if (Array.isArray(item.ingredients)) {
    specsList = item.ingredients
  }

  // Resolve steps
  const stepsList =
    Array.isArray(item.steps) && item.steps.length > 0
      ? item.steps
      : ['Verify firearm is completely unloaded and point
in safe direction.', 'Inspect receiver and mechanism for
maintenance.']

  return {
    ...item,
    id: item.id,
    name: item.name || 'Hardware Piece',
    type: item.type || item.category || 'Hardware',
    category: item.type || item.category || 'Hardware',
    caliber: item.caliber || 'Standard',
    price: Number(item.price ?? item.cook_time ?? 0),
    description: item.description || '',
    image_url: item.image_url || item.image ||
```

```
'/guns/pistol.svg',
    image: item.image_url || item.image ||
'/guns/pistol.svg',
    ingredients: specsList,
    steps: stepsList,
  }
}

//
------------------------------------------------------------
-----------------
// GunService — wraps backend CRUD endpoints.
// Supports both Supabase PostgREST table
(/rest/v1/products) and
// standard REST API (/api/v1/recipes).
//
------------------------------------------------------------
-----------------
class GunService {
  /**
   * Fetch a paginated, filterable list of products.
   */
  async getGuns(params = {}) {
    if (IS_SUPABASE) {
      const queryParts = ['select=*']

      if (params.search) {

queryParts.push(`name=ilike.*${encodeURIComponent(params.s
earch)}*`)
      }

      if (params.category && params.category !== 'All') {

queryParts.push(`type=eq.${encodeURIComponent(params.categ
ory)}`)
      }

      if (params.sort_by) {
        const order = params.order === 'desc' ? 'desc' :
'asc'
        queryParts.push(`order=
encodeURIComponent(params.sort_by).{order}`)
      } else {
        queryParts.push('order=created_at.desc')
      }

      if (params.limit) {
        queryParts.push(`limit=${params.limit}`)
      }

      const queryString = queryParts.join('&')
      const data = await
apiClient.get(`/rest/v1/products?${queryString}`)
      const list = Array.isArray(data) ?
```

```
data.map(normalizeProduct) : []

      return {
        success: true,
        data: list,
        pagination: {
          total: list.length,
          page: params.page || 1,
          limit: params.limit || 50,
        },
      }
    }

    // Fallback: university REST API
    const clean = Object.fromEntries(
      Object.entries(params).filter(([, v]) => v !== '' &&
v != null),
    )
    return apiClient.get('/api/v1/recipes', { params:
clean })
  }

  /**
   * Fetch a single product by ID.
   */
  async getGunById(id) {
    if (IS_SUPABASE) {
      const data = await
apiClient.get(`/rest/v1/products?id=eq.${id}&select=*`)
      if (Array.isArray(data) && data.length > 0) {
        return { success: true, data:
normalizeProduct(data[0]) }
      }
      throw new Error('Hardware item not found.')
    }
    return apiClient.get(`/api/v1/recipes/${id}`)
  }

  /**
   * Create a new product.
   */
  async createGun(data) {
    if (IS_SUPABASE) {
      const specsArray = Array.isArray(data.ingredients)
        ? data.ingredients.map((i) => `${i.name}:
${i.quantity}`)
        : Array.isArray(data.specs)
        ? data.specs
        : []

      const payload = {
        name: data.name,
        type: data.type || (data.category === 'minuman' ?
'Rifle' : 'Pistol'),
        caliber: data.caliber || '9mm',
```

```
        price: Number(data.price ?? data.cook_time ?? 0),
        description: data.description || '',
        specs: specsArray,
        image_url: data.image_url || null,
      }

      const res = await
apiClient.post('/rest/v1/products', payload, {
        headers: { Prefer: 'return=representation' },
      })
      const created = Array.isArray(res) ? res[0] : res
      return { success: true, data:
normalizeProduct(created) }
    }
    return apiClient.post('/api/v1/recipes', data)
  }

  /**
   * Update existing product.
   */
  async updateGun(id, data) {
    if (IS_SUPABASE) {
      const specsArray = Array.isArray(data.ingredients)
        ? data.ingredients.map((i) => `${i.name}:
${i.quantity}`)
        : Array.isArray(data.specs)
        ? data.specs
        : undefined

      const payload = {
        updated_at: new Date().toISOString(),
      }
      if (data.name !== undefined) payload.name =
data.name
      if (data.type !== undefined) payload.type =
data.type
      if (data.caliber !== undefined) payload.caliber =
data.caliber
      if (data.price !== undefined || data.cook_time !==
undefined) {
        payload.price = Number(data.price ??
data.cook_time ?? 0)
      }
      if (data.description !== undefined)
payload.description = data.description
      if (specsArray !== undefined) payload.specs =
specsArray
      if (data.image_url !== undefined) payload.image_url
= data.image_url

      const res = await
apiClient.patch(`/rest/v1/products?id=eq.${id}`, payload,
{
        headers: { Prefer: 'return=representation' },
      })
```

```
      const updated = Array.isArray(res) ? res[0] : res
      return { success: true, data:
normalizeProduct(updated) }
    }
    return apiClient.put(`/api/v1/recipes/${id}`, data)
  }

  /**
   * Partial update (PATCH).
   */
  async patchGun(id, data) {
    if (IS_SUPABASE) {
      return this.updateGun(id, data)
    }
    return apiClient.patch(`/api/v1/recipes/${id}`, data)
  }

  /**
   * Delete product.
   */
  async deleteGun(id) {
    if (IS_SUPABASE) {
      await
apiClient.delete(`/rest/v1/products?id=eq.${id}`)
      return { success: true }
    }
    return apiClient.delete(`/api/v1/recipes/${id}`)
  }
}

export default new GunService()
```

src/services/reviewService.js

```
import { apiClient, IS_SUPABASE } from '../config/api'

//
-----------------------------------------------------------
-----------------
// ReviewService — product reviews & ratings.
// Supports both Supabase (public.reviews) and university
API (/api/v1/recipes/:id/reviews).
//
-----------------------------------------------------------
-----------------

class ReviewService {
  /**
   * Fetch all reviews for a given product.
   */
  async getReviews(gunId) {
    if (IS_SUPABASE) {
```

```
      const data = await apiClient.get(
`/rest/v1/reviews?product_id=eq.${gunId}&select=*&order=cr
eated_at.desc`,
      )
      const list = Array.isArray(data)
        ? data.map((r) => ({
            id: r.id,
            product_id: r.product_id,
            user_identifier: r.user_id ? `Operator
${String(r.user_id).substring(0, 8)}` : 'Anonymous',
            rating: r.rating,
            comment: r.comment || '',
            created_at: r.created_at,
          }))
        : []

      return { success: true, data: list }
    }

    return
apiClient.get(`/api/v1/recipes/${gunId}/reviews`)
  }

  /**
   * Submit a new review.
   * @param {string|number} gunId
   * @param {Object} reviewData  { user_identifier,
rating, comment }
   */
  async createReview(gunId, reviewData) {
    if (IS_SUPABASE) {
      const payload = {
        product_id: Number(gunId),
        user_id: reviewData.user_id ||
reviewData.user_identifier,
        rating: Number(reviewData.rating),
        comment: reviewData.comment || '',
      }

      const res = await apiClient.post('/rest/v1/reviews',
payload, {
        headers: { Prefer: 'return=representation' },
      })
      const created = Array.isArray(res) ? res[0] : res
      return { success: true, data: created }
    }

    return
apiClient.post(`/api/v1/recipes/${gunId}/reviews`,
reviewData)
  }

  /**
   * Edit an existing review.
```

```
   */
  async updateReview(reviewId, reviewData) {
    if (IS_SUPABASE) {
      const res = await apiClient.patch(
        `/rest/v1/reviews?id=eq.${reviewId}`,
        {
          rating: Number(reviewData.rating),
          comment: reviewData.comment,
        },
        { headers: { Prefer: 'return=representation' } },
      )
      return { success: true, data: Array.isArray(res) ?
res[0] : res }
    }

    return apiClient.put(`/api/v1/reviews/${reviewId}`,
reviewData)
  }

  /**
   * Delete a review.
   */
  async deleteReview(reviewId) {
    if (IS_SUPABASE) {
      await
apiClient.delete(`/rest/v1/reviews?id=eq.${reviewId}`)
      return { success: true }
    }

    return apiClient.delete(`/api/v1/reviews/${reviewId}`)
  }
}

export default new ReviewService()
```

src/services/uploadService.js

```
import axios from 'axios'
import { BASE_URL, SUPABASE_KEY, IS_SUPABASE } from
'../config/api'

//
----------------------------------------------------------
----------------
// UploadService — image upload service.
// Supports Supabase Storage
(/storage/v1/object/:bucket/:file) and
// standard REST API (/api/v1/upload), with client-side
fallback.
//
----------------------------------------------------------
----------------
```

```
const ALLOWED_TYPES = ['image/jpeg', 'image/jpg',
'image/png', 'image/webp']
const MAX_SIZE_BYTES = 5 * 1024 * 1024 // 5 MB

class UploadService {
  /**
   * Upload a single image file.
   * @param {File} file
   * @returns {Promise<{ success: boolean, data: { url:
string } }>}
   */
  async uploadImage(file) {
    if (!file) {
      throw new Error('No file provided.')
    }
    if (!ALLOWED_TYPES.includes(file.type)) {
      throw new Error(
        'Invalid file type. Allowed formats: .jpg, .jpeg,
.png, .webp',
      )
    }
    if (file.size > MAX_SIZE_BYTES) {
      throw new Error('File size exceeds the 5 MB limit.')
    }

    // --- Supabase Storage flow ---
    if (IS_SUPABASE) {
      const ext = file.name.split('.').pop() || 'png'
      const fileName =
`gun_${Date.now()}_${Math.random().toString(36).substring(2
, 7)}.${ext}`
      const bucketName = 'guns'

      try {
        const uploadUrl = `BASE_URL/storage/v1/object/
{bucketName}/${fileName}`
        await axios.post(uploadUrl, file, {
          headers: {
            'Content-Type': file.type,
            apikey: SUPABASE_KEY,
            Authorization: `Bearer ${SUPABASE_KEY}`,
          },
          timeout: 30000,
        })

        const publicUrl = `BASE_URL/storage/v1/object/public/
{bucketName}/${fileName}`
        return {
          success: true,
          data: { url: publicUrl, filename: fileName },
        }
      } catch (err) {
        console.warn('Supabase bucket upload not
```

```
accessible, creating base64 data URL:', err)
        // Read file as base64 Data URL so the image
displays and persists
        const dataUrl = await new Promise((resolve, reject)
=> {
          const reader = new FileReader()
          reader.onload = () => resolve(reader.result)
          reader.onerror = reject
          reader.readAsDataURL(file)
        })

        return {
          success: true,
          data: { url: dataUrl, filename: file.name },
        }
      }
    }

    // --- University API flow (/api/v1/upload) ---
    const formData = new FormData()
    formData.append('image', file)

    const response = await
axios.post(`${BASE_URL}/api/v1/upload`, formData, {
      headers: { 'Content-Type': 'multipart/form-data' },
      timeout: 30000,
    })

    return response.data
  }
}

export default new UploadService()
```

src/services/[userService.js]

```
//
----------------------------------------------------------
----------------
// User identity — generates and persists a unique client
identifier in
// localStorage so the Favorites and Reviews APIs can
associate data with
// this browser session.  No authentication is involved.
//
----------------------------------------------------------
----------------

const USER_IDENTIFIER_KEY = 'bore_barrel_user_id'
const USER_PROFILE_KEY = 'bore_barrel_user_profile'

/**
```

```
 * Check if a string is a valid UUID
 */
const isValidUUID = (str) =>
  typeof str === 'string' &&

/^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3
}-[0-9a-f]{12}$/i.test(str)

/**
 * Generate a standard RFC4122 v4 UUID
 */
const generateUUID = () => {
  if (typeof crypto !== 'undefined' && crypto.randomUUID) {
    return crypto.randomUUID()
  }
  return
'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, (c)
=> {
    const r = (Math.random() * 16) | 0
    const v = c === 'x' ? r : (r & 0x3) | 0x8
    return v.toString(16)
  })
}

/**
 * Return a stable, randomly-generated user identifier
(UUID).
 * Compatible with Supabase PostgreSQL UUID foreign keys.
 * Creates one on first call and stores it in localStorage.
 */
export const getUserIdentifier = () => {
  try {
    let userId = localStorage.getItem(USER_IDENTIFIER_KEY)
    if (!userId || !isValidUUID(userId)) {
      userId = generateUUID()
      localStorage.setItem(USER_IDENTIFIER_KEY, userId)
    }
    return userId
  } catch {
    // Private / incognito mode may throw — return a
transient UUID.
    return generateUUID()
  }
}

/**
 * Return the user profile object stored in localStorage,
or a sensible
 * default.  The profile is purely cosmetic (username for
reviews, etc.).
 */
export const getUserProfile = () => {
  try {
    const raw = localStorage.getItem(USER_PROFILE_KEY)
    if (raw) return JSON.parse(raw)
```

```
  } catch {
    /* corrupted data — fall through to default */
  }
  return {
    username: 'Shooter',
    avatar: null,
    bio: '',
    userId: getUserIdentifier(),
  }
}

/**
 * Persist an updated profile object.
 */
export const saveUserProfile = (profile) => {
  try {
    const data = {
      ...profile,
      userId: getUserIdentifier(),
      updatedAt: new Date().toISOString(),
    }
    localStorage.setItem(USER_PROFILE_KEY,
JSON.stringify(data))
    return { success: true, data }
  } catch (err) {
    return { success: false, message: err.message }
  }
}

/**
 * Update username
 */
export const updateUsername = (username) => {
  const profile = getUserProfile()
  profile.username = username.trim() || 'Shooter'
  return saveUserProfile(profile)
}

/**
 * Update bio
 */
export const updateBio = (bio) => {
  const profile = getUserProfile()
  profile.bio = bio.trim()
  return saveUserProfile(profile)
}

/**
 * Update avatar
 */
export const updateAvatar = (avatarBase64) => {
  const profile = getUserProfile()
  profile.avatar = avatarBase64
  return saveUserProfile(profile)
}
```

```
/**
 * Reset identifier (generates a new anonymous identifier)
 */
export const resetUserIdentifier = () => {
  try {
    localStorage.removeItem(USER_IDENTIFIER_KEY)
    localStorage.removeItem(USER_PROFILE_KEY)
    return getUserIdentifier()
  } catch {
    return `anon_${Date.now()}`
  }
}

export default {
  getUserIdentifier,
  getUserProfile,
  saveUserProfile,
  updateUsername,
  updateBio,
  updateAvatar,
  resetUserIdentifier,
}
```

5. Lanjut ke *folder* src/pages kemudian edit tiap *file* yang ada didalamnya dengan kode di bawah ini

src\pages\CreateGunPage.jsx

```
import { useState, useEffect, useCallback } from 'react'
import {
  ArrowLeft,
  Upload,
  X,
  Plus,
  Loader,
  Save,
  CheckCircle,
  AlertCircle,
} from 'lucide-react'
import gunService from '../services/gunService'
import uploadService from '../services/uploadService'
import {
  saveDraft,
  loadDraft,
  deleteDraft,
  hasDraft,
  getDraftTimestamp,
  formatDraftTime,
} from '../utils/draftStorage'
import                    ConfirmModal                    from
'../components/modals/ConfirmModal'
```

```
/**
 * CreateGunPage — Create a new weapon / hardware item with image upload,
 * draft persistence (auto-save), specs, and assembly steps.
 */
export default function CreateGunPage({ onBack, onSuccess }) {
  // Step state: 'upload' (upload or choose image) or 'form' (enter details)
  const [currentStep, setCurrentStep] = useState('upload')

  const [formData, setFormData] = useState({
    name: '',
    type: 'Pistol',
    caliber: '9mm',
      category: 'makanan', // API validation expects 'makanan' or 'minuman'
    difficulty: 'mudah', // 'mudah' (Standard) | 'sedang' (Service) | 'sulit' (Match)
    price: 599,
    description: '',
    servings: 1,
    is_featured: false,
  })

  const [imageFile, setImageFile] = useState(null)
  const [imagePreview, setImagePreview] = useState(null)
    const [uploadedImageUrl, setUploadedImageUrl] = useState('')

  // Specifications (ingredients in API)
  const [specs, setSpecs] = useState([
    { name: 'Frame', quantity: 'Polymer' },
    { name: 'Action', quantity: 'Striker-Fired' },
    { name: 'Capacity', quantity: '17 rounds' },
  ])

  // Maintenance / Assembly Steps (steps in API)
  const [steps, setSteps] = useState([
    'Verify firearm is completely unloaded and point in safe direction.',
    'Disassemble slide assembly from receiver frame for routine maintenance.',
  ])

  const [uploading, setUploading] = useState(false)
  const [creating, setCreating] = useState(false)
  const [error, setError] = useState('')
  const [successMsg, setSuccessMsg] = useState('')

  // Draft management
  const [showDraftModal, setShowDraftModal] = useState(() => hasDraft('create'))
  const [draftTimestamp, setDraftTimestamp] = useState(() => getDraftTimestamp('create'))
```

```
  const handleSaveDraft = useCallback((notify = true) => {
    const draftData = {
      formData,
      specs,
      steps,
      uploadedImageUrl,
    }
    const saved = saveDraft(draftData, 'create')
    if (saved) {
      const ts = getDraftTimestamp('create')
      setDraftTimestamp(ts)
      if (notify) {
                                setSuccessMsg(`Draft    saved
(${formatDraftTime(ts)})`)
        setTimeout(() => setSuccessMsg(''), 3000)
      }
    }
  }, [formData, specs, steps, uploadedImageUrl])

  // Auto-save draft every 30 seconds while in form step
  useEffect(() => {
    if (currentStep !== 'form') return

    const interval = setInterval(() => {
      handleSaveDraft(false)
    }, 30000)

    return () => clearInterval(interval)
  }, [handleSaveDraft, currentStep])

  const handleLoadDraft = () => {
    const draft = loadDraft('create')
    if (draft) {
      if (draft.formData) setFormData(draft.formData)
      if (draft.specs) setSpecs(draft.specs)
      if (draft.steps) setSteps(draft.steps)
      if (draft.uploadedImageUrl) {
        setUploadedImageUrl(draft.uploadedImageUrl)
        setImagePreview(draft.uploadedImageUrl)
        setCurrentStep('form')
      }
      setShowDraftModal(false)
      setSuccessMsg('Draft successfully loaded.')
      setTimeout(() => setSuccessMsg(''), 4000)
    }
  }

  const handleDiscardDraft = () => {
    deleteDraft('create')
    setShowDraftModal(false)
    setDraftTimestamp(null)
  }

  // Handle file selection
  const handleImageChange = (e) => {
```

```
    const file = e.target.files[0]
    if (!file) return

    if (file.size > 5 * 1024 * 1024) {
      setError('File size exceeds 5MB limit.')
      return
    }

          const allowed = ['image/jpeg', 'image/jpg',
'image/png', 'image/webp']
    if (!allowed.includes(file.type)) {
          setError('Allowed formats: .jpg, .jpeg, .png,
.webp')
      return
    }

    setImageFile(file)
    setImagePreview(URL.createObjectURL(file))
    setError('')
  }

  // Quick fallback select vector from local assets
  const handleSelectPreset = (url) => {
    setImageFile(null)
    setImagePreview(url)
    setUploadedImageUrl(url)
    setCurrentStep('form')
    setError('')
  }

  // Upload image to server
  const handleUploadImage = async () => {
    if (!imageFile) {
       // If user already previewed a preset or entered a
url, advance to form
      if (uploadedImageUrl) {
        setCurrentStep('form')
        return
      }
      setError('Please select an image file first.')
      return
    }

    try {
      setUploading(true)
      setError('')
                          const result = await
uploadService.uploadImage(imageFile)
      if (result.success && result.data?.url) {
        setUploadedImageUrl(result.data.url)
        setCurrentStep('form')
      } else {
          throw new Error(result.message || 'Image upload
failed.')
      }
    } catch (err) {
```

```
        console.warn('Upload API failed, falling back to
local preview:', err)
       // Allow fallback to image preview so the user can
continue even if MinIO bucket is unreachable
                    setUploadedImageUrl(imagePreview   ||
'/guns/pistol.svg')
      setCurrentStep('form')
    } finally {
      setUploading(false)
    }
  }

  const handleFormChange = (e) => {
    const { name, value, type, checked } = e.target
    setFormData((prev) => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value,
    }))
  }

  // Specs handlers
  const handleSpecChange = (index, field, value) => {
    setSpecs((prev) => {
      const copy = [...prev]
      copy[index] = { ...copy[index], [field]: value }
      return copy
    })
  }

  const addSpec = () => {
     setSpecs((prev) => [...prev, { name: '', quantity: ''
}])
  }

  const removeSpec = (index) => {
    if (specs.length <= 1) return
    setSpecs((prev) => prev.filter((_, i) => i !== index))
  }

  // Steps handlers
  const handleStepChange = (index, value) => {
    setSteps((prev) => {
      const copy = [...prev]
      copy[index] = value
      return copy
    })
  }

  const addStep = () => {
    setSteps((prev) => [...prev, ''])
  }

  const removeStep = (index) => {
    if (steps.length <= 1) return
    setSteps((prev) => prev.filter((_, i) => i !== index))
  }
```

```
  // Submit create product
  const handleSubmit = async (e) => {
    e.preventDefault()
    setError('')

                        if      (!formData.name.trim()      ||
formData.name.trim().length < 3) {
             setError('Product  name  must  be  at  least  3
characters.')
      return
    }

    const cleanSpecs = specs
      .filter((s) => s.name.trim())
      .map((s) => ({
        name: s.name.trim(),
        quantity: s.quantity.trim() || 'Standard',
      }))

    if (cleanSpecs.length === 0) {
                setError('Please   provide   at   least   one
specification.')
      return
    }

    const cleanSteps = steps
      .filter((s) => s.trim())
      .map((s) => s.trim())

    if (cleanSteps.length === 0) {
       setError('Please provide at least one maintenance /
assembly step.')
      return
    }

          //  Prepare  payload  compliant  with  backend
/api/v1/recipes schema
    const payload = {
      name: formData.name.trim(),
      category: formData.category || 'makanan',
              description:  formData.description.trim()  ||
`${formData.type} caliber ${formData.caliber}.`,
      image_url: uploadedImageUrl || '/guns/pistol.svg',
      prep_time: 15,
        cook_time: Number(formData.price) || 500, // Price
stored in cook_time
      servings: Number(formData.servings) || 1,
      difficulty: formData.difficulty || 'mudah',
      is_featured: Boolean(formData.is_featured),
      ingredients: cleanSpecs,
      steps: cleanSteps,
    }

    try {
      setCreating(true)
```

```
      const response = await gunService.createGun(payload)
      if (response.success || response.data) {
        deleteDraft('create')
        if (onSuccess) {
          onSuccess(response.data)
        } else if (onBack) {
          onBack()
        }
      } else {
          throw new Error(response.message || 'Failed to
create product.')
      }
    } catch (err) {
       setError(err.message || 'Error occurred while saving
product to server.')
    } finally {
      setCreating(false)
    }
  }

  return (
    <div className="form-page">
      {/* Header */}
      <div className="form-head">
            <button type="button" className="detail-back"
onClick={onBack}>
          <ArrowLeft size={18} /> Back to Catalog
        </button>
        <div className="form-head-actions">
          {currentStep === 'form' && (
            <button
              type="button"
              className="btn btn--outline btn--sm"
              onClick={() => handleSaveDraft(true)}
              title="Save draft"
            >
              <Save size={14} /> Save Draft
            </button>
          )}
        </div>
      </div>

      <div className="form-title-group">
        <h1 className="display">Register New Hardware</h1>
        <p className="lede">
           Add a firearm or accessory to the catalog. Data
will be synchronized with the
          university RESTful backend.
        </p>
      </div>

      {/* Notifications */}
      {error && (
        <div className="toast toast--error">
          <AlertCircle size={18} /> {error}
        </div>
```

```
      )}
      {successMsg && (
        <div className="toast toast--success">
          <CheckCircle size={18} /> {successMsg}
        </div>
      )}

      {/* Step Indicators */}
      <div className="form-steps-nav">
            <div  className={`step-badge  ${currentStep  ===
'upload' ? 'step-badge--active' : 'step-badge--done'}`}>
          1. Imagery
        </div>
        <div className="step-badge-connector" />
            <div  className={`step-badge  ${currentStep  ===
'form' ? 'step-badge--active' : ''}`}>
          2. Specifications &amp;amp; Details
        </div>
      </div>

      {/* STEP 1: Upload Image */}
      {currentStep === 'upload' && (
        <div className="card-box">
                    <h2  className="card-box-title">Weapon
Imagery</h2>
          <p className="card-box-desc">
               Upload an authentic photograph or blueprint
(max 5MB), or select an armory preset.
          </p>

          {!imagePreview ? (
            <div className="upload-dropzone">
              <input
                id="gun-image-input"
                type="file"

accept="image/jpeg,image/jpg,image/png,image/webp"
                onChange={handleImageChange}
                className="hidden-file-input"
              />
                            <label  htmlFor="gun-image-input"
className="upload-label">
                  <Upload size={36} className="upload-icon"
/>
                   <span className="upload-main-text">Click
to choose an image file</span>
                    <span  className="upload-sub-text">JPG,
PNG, or WebP up to 5MB</span>
              </label>

              <div className="preset-selector">
                   <span  className="preset-title">Or use a
stock preset:</span>
                <div  className="preset-pills">
                  <button
                    type="button"
```

```
                    className="pill"
                                                onClick={()  =>
handleSelectPreset('/guns/pistol.svg')}
                  >
                    Pistol Vector
                  </button>
                  <button
                    type="button"
                    className="pill"
                                                onClick={()  =>
handleSelectPreset('/guns/rifle.svg')}
                  >
                    Rifle Vector
                  </button>
                  <button
                    type="button"
                    className="pill"
                                                onClick={()  =>
handleSelectPreset('/guns/shotgun.svg')}
                  >
                    Shotgun Vector
                  </button>
                </div>
              </div>
            </div>
          ) : (
            <div className="preview-container">
              <div className="preview-image-wrap">
                    <img src={imagePreview} alt="Hardware
preview" className="preview-img" />
                <button
                  type="button"
                  className="preview-remove-btn"
                  onClick={() => {
                    setImageFile(null)
                    setImagePreview(null)
                    setUploadedImageUrl('')
                  }}
                  title="Remove image"
                >
                  <X size={18} />
                </button>
              </div>

              <div className="preview-actions">
                <button
                  type="button"
                  className="btn btn--primary"
                  onClick={handleUploadImage}
                  disabled={uploading}
                >
                  {uploading ? (
                    <>
                         <Loader size={18} className="spin"
/> Uploading to MinIO…
                    </>
```

```
                  ) : (
                    <>
                       <Upload size={18} /> Confirm Imagery
&amp;amp; Continue
                    </>
                  )}
                </button>
                <button
                  type="button"
                  className="btn btn--outline"
                  onClick={() => {
                    setUploadedImageUrl(imagePreview)
                    setCurrentStep('form')
                  }}
                >
                  Continue without uploading
                </button>
              </div>
            </div>
          )}
        </div>
      )}

      {/* STEP 2: Product Form */}
      {currentStep === 'form' && (
                           <form   onSubmit={handleSubmit}
className="entry-form">
          {/* Active Image Thumbnail */}
          <div className="selected-thumb-bar">
               <img src={uploadedImageUrl || imagePreview}
alt="Selected" className="thumb-small" />
            <div className="thumb-info">
                     <span  className="thumb-title">Imagery
Assigned</span>
                     <span  className="thumb-sub">Ready  for
registration</span>
            </div>
            <button
              type="button"
              className="btn btn--outline btn--sm"
              onClick={() => setCurrentStep('upload')}
            >
              Change Image
            </button>
          </div>

          {/* Core Info */}
          <div className="form-grid">
            <div className="form-group">
                                      <label   htmlFor="name"
className="form-label">
                                   Model  Designation  <span
className="req">*</span>
              </label>
              <input
                id="name"
```

```
                type="text"
                name="name"
                value={formData.name}
                onChange={handleFormChange}
                    placeholder="e.g. Glock 19 Gen 5, Colt
M4A1..."
                className="form-input"
                required
              />
            </div>

            <div className="form-group">
                                    <label   htmlFor="type"
className="form-label">
                                Platform  /  Category  <span
className="req">*</span>
              </label>
              <select
                id="type"
                name="type"
                value={formData.type}
                onChange={(e) => {
                  handleFormChange(e)
                  // Map category for backend schema
                  setFormData((prev) => ({
                    ...prev,
                      category: e.target.value === 'Pistol'
? 'makanan' : 'minuman',
                    }))
                }}
                className="form-select"
              >
                                  <option  value="Pistol">Pistol
(Sidearm)</option>
                              <option  value="Rifle">Rifle  (Long
Gun)</option>
                                <option  value="Shotgun">Shotgun
(Smoothbore)</option>
                    <option value="Submachine Gun">Submachine
Gun</option>
              </select>
            </div>
          </div>

          <div className="form-grid form-grid--3">
            <div className="form-group">
                                      <label  htmlFor="caliber"
className="form-label">
                                  Caliber  /  Chambering  <span
className="req">*</span>
              </label>
              <input
                id="caliber"
                type="text"
                name="caliber"
                value={formData.caliber}
```

```
                onChange={handleFormChange}
                placeholder="e.g. 9mm, 5.56mm, 12 Gauge"
                className="form-input"
                required
              />
            </div>

            <div className="form-group">
                                    <label  htmlFor="price"
className="form-label">
                Price (USD) <span className="req">*</span>
              </label>
              <input
                id="price"
                type="number"
                name="price"
                value={formData.price}
                onChange={handleFormChange}
                placeholder="599"
                min="0"
                className="form-input"
                required
              />
            </div>

            <div className="form-group">
                                <label  htmlFor="difficulty"
className="form-label">
                               Grade  /  Build  Quality  <span
className="req">*</span>
              </label>
              <select
                id="difficulty"
                name="difficulty"
                value={formData.difficulty}
                onChange={handleFormChange}
                className="form-select"
              >
                                 <option  value="mudah">Commercial  /
Standard</option>
                   <option value="sedang">Mil-Spec / Service
Issue</option>
                                <option  value="sulit">Match  Grade  /
Custom</option>
              </select>
            </div>
          </div>

          <div className="form-group">
                                  <label  htmlFor="description"
className="form-label">
              Armory Description
            </label>
            <textarea
              id="description"
              name="description"
```

```
              value={formData.description}
              onChange={handleFormChange}
                      placeholder="Operational  background,
ergonomics, intended mission profile..."
              rows={3}
              className="form-textarea"
            />
          </div>

          {/* Specifications (Ingredients) */}
          <div className="form-fieldset">
            <div className="fieldset-header">
               <label className="form-label">Specifications
&amp;amp; Components</label>
              <button
                type="button"
                className="btn btn--outline btn--sm"
                onClick={addSpec}
              >
                <Plus size={14} /> Add Spec
              </button>
            </div>
            <div className="multi-rows">
              {specs.map((spec, i) => (
                <div key={i} className="multi-row">
                  <input
                    type="text"
                       placeholder="Attribute (e.g. Barrel
Length, Action)"
                    value={spec.name}
                       onChange={(e) => handleSpecChange(i,
'name', e.target.value)}
                    className="form-input"
                  />
                  <input
                    type="text"
                         placeholder="Value (e.g. 16 inch,
Striker-fired)"
                    value={spec.quantity}
                       onChange={(e) => handleSpecChange(i,
'quantity', e.target.value)}
                    className="form-input"
                  />
                  <button
                    type="button"
                    className="btn btn--danger-ghost"
                    onClick={() => removeSpec(i)}
                    disabled={specs.length <= 1}
                    title="Remove item"
                  >
                    <X size={16} />
                  </button>
                </div>
              ))}
            </div>
          </div>
```

```
          {/* Assembly / Maintenance Steps */}
          <div className="form-fieldset">
            <div className="fieldset-header">
               <label className="form-label">Assembly &amp;
Maintenance Protocol</label>
              <button
                type="button"
                className="btn btn--outline btn--sm"
                onClick={addStep}
              >
                <Plus size={14} /> Add Step
              </button>
            </div>
            <div className="multi-rows">
              {steps.map((st, i) => (
                <div key={i} className="step-row">
                            <span  className="step-num">{i  +
1}</span>
                  <input
                    type="text"
                       placeholder="e.g. Clear the chamber
and lock slide back..."
                    value={st}
                      onChange={(e) => handleStepChange(i,
e.target.value)}
                    className="form-input"
                  />
                  <button
                    type="button"
                    className="btn btn--danger-ghost"
                    onClick={() => removeStep(i)}
                    disabled={steps.length <= 1}
                    title="Remove step"
                  >
                    <X size={16} />
                  </button>
                </div>
              ))}
            </div>
          </div>

          {/* Featured checkbox */}
          <div className="checkbox-row">
            <input
              id="is_featured"
              type="checkbox"
              name="is_featured"
              checked={formData.is_featured}
              onChange={handleFormChange}
              className="form-checkbox"
            />
                              <label  htmlFor="is_featured"
className="checkbox-label">
                    Feature in Armory Showcase (Highlight
product on catalog)
```

```
            </label>
          </div>

          {/* Submit bar */}
          <div className="form-actions-bar">
            <button
              type="button"
              className="btn btn--outline"
              onClick={onBack}
              disabled={creating}
            >
              Cancel
            </button>
            <button
              type="submit"
              className="btn btn--primary"
              disabled={creating}
            >
              {creating ? (
                <>
                      <Loader size={18} className="spin" />
Registering Hardware…
                </>
              ) : (
                'Commit to Armory'
              )}
            </button>
          </div>
        </form>
      )}

      {/* Restore Draft Modal */}
      <ConfirmModal
        open={showDraftModal}
        title="Resume Previous Draft?"
                    message={`An unfinished draft from
${draftTimestamp ? formatDraftTime(draftTimestamp) :
'earlier'} was detected. Would you like to resume
editing?`}
        confirmLabel="Load Draft"
        cancelLabel="Discard"
        onConfirm={handleLoadDraft}
        onCancel={handleDiscardDraft}
      />
    </div>
  )
}
```

src\pages\EditGunPage.jsx

```
import { useState, useEffect } from 'react'
import {
  ArrowLeft,
  Upload,
  X,
```

```
  Plus,
  Loader,
  Save,
  CheckCircle,
  AlertCircle,
} from 'lucide-react'
import gunService from '../services/gunService'
import uploadService from '../services/uploadService'
import {
  saveDraft,
  loadDraft,
  deleteDraft,
  hasDraft,
  getDraftTimestamp,
  formatDraftTime,
} from '../utils/draftStorage'
import                    ConfirmModal                    from
'../components/modals/ConfirmModal'

/**
 * EditGunPage — Edit an existing weapon / hardware item.
 * Supports updating specs, steps, image replacement, and
draft management.
 */
export default function EditGunPage({ gun, gunId, onBack,
onSuccess }) {
  const targetId = gunId || gun?.id

  const [loading, setLoading] = useState(true)
  const [updating, setUpdating] = useState(false)
  const [error, setError] = useState('')
  const [successMsg, setSuccessMsg] = useState('')

  const [formData, setFormData] = useState({
    name: '',
    type: 'Pistol',
    caliber: '9mm',
    category: 'makanan',
    difficulty: 'mudah',
    price: 599,
    description: '',
    servings: 1,
    is_featured: false,
  })

       const   [currentImageUrl,   setCurrentImageUrl]   =
useState('')
  const [imageFile, setImageFile] = useState(null)
  const [imagePreview, setImagePreview] = useState(null)
  const [uploading, setUploading] = useState(false)

  // Specs & Steps
     const  [specs,  setSpecs]  =  useState([{  name:  '',
quantity: '' }])
  const [steps, setSteps] = useState([''])
```

```
  // Draft modal
   const [showDraftModal, setShowDraftModal] = useState(()
=> (targetId ? hasDraft(targetId) : false))
   const [draftTimestamp, setDraftTimestamp] = useState(()
=> (targetId ? getDraftTimestamp(targetId) : null))

  // Fetch initial data
  useEffect(() => {
    const fetchGun = async () => {
      try {
        setLoading(true)
        setError('')

             // If gun was passed directly and has full
details, use it; else fetch from API
        let data = gun
        if (!data || !data.ingredients || !data.steps) {
                                        if    (targetId    &&
!String(targetId).startsWith('local-')) {
                                      const   res   =   await
gunService.getGunById(targetId)
            if (res.success && res.data) {
              data = res.data
            }
          }
        }

        if (data) {
          setFormData({
            name: data.name || '',
                    type: data.type  ||  (data.category  ===
'makanan' ? 'Pistol' : 'Rifle'),
            caliber: data.caliber || '9mm',
            category: data.category || 'makanan',
            difficulty: data.difficulty || 'mudah',
            price: data.price ?? data.cook_time ?? 599,
            description: data.description || '',
            servings: data.servings || 1,
            is_featured: Boolean(data.is_featured),
          })
           setCurrentImageUrl(data.image_url || data.image
|| '/guns/pistol.svg')

           if (data.ingredients && data.ingredients.length
> 0) {
            setSpecs(data.ingredients.map((ing) => ({
              name: ing.name || '',
              quantity: ing.quantity || '',
            })))
          } else {
            setSpecs([
                { name: 'Platform', quantity: data.type ||
'Pistol' },
               { name: 'Caliber', quantity: data.caliber ||
'9mm' },
            ])
```

```
          }

          if (data.steps && data.steps.length > 0) {
             setSteps(data.steps.map((st) => (typeof st ===
'string' ? st : st.instruction || '')))
          } else {
               setSteps(['Standard manufacturer maintenance
and cleaning protocol.'])
          }

        } else {
          setError('Hardware record could not be loaded.')
        }
      } catch (err) {
          setError(err.message || 'Failed to load hardware
record from server.')
      } finally {
        setLoading(false)
      }
    }

    if (targetId) {
      fetchGun()
    }
                            //       eslint-disable-next-line
react-hooks/exhaustive-deps
  }, [targetId])

  // Draft operations
  const handleLoadDraft = () => {
    const draft = loadDraft(targetId)
    if (draft) {
      if (draft.formData) setFormData(draft.formData)
      if (draft.specs) setSpecs(draft.specs)
      if (draft.steps) setSteps(draft.steps)
      if (draft.imagePreview) {
        setImagePreview(draft.imagePreview)
      }
      setShowDraftModal(false)
      setSuccessMsg('Saved draft restored.')
      setTimeout(() => setSuccessMsg(''), 3000)
    }
  }

  const handleDiscardDraft = () => {
    deleteDraft(targetId)
    setShowDraftModal(false)
    setDraftTimestamp(null)
  }

  const handleManualSaveDraft = () => {
    const draftData = {
      formData,
      specs,
      steps,
      imagePreview: imagePreview || currentImageUrl,
```

```
    }
    const saved = saveDraft(draftData, targetId)
    if (saved) {
      const ts = getDraftTimestamp(targetId)
      setDraftTimestamp(ts)
                                setSuccessMsg(`Draft     saved
(${formatDraftTime(ts)})`)
      setTimeout(() => setSuccessMsg(''), 3000)
    }
  }

  // Handle new image file
  const handleImageChange = (e) => {
    const file = e.target.files[0]
    if (!file) return

    if (file.size > 5 * 1024 * 1024) {
      setError('File size exceeds 5MB limit.')
      return
    }

          const   allowed   =   ['image/jpeg',   'image/jpg',
'image/png', 'image/webp']
    if (!allowed.includes(file.type)) {
           setError('Allowed  formats:  .jpg,  .jpeg,  .png,
.webp')
      return
    }

    setImageFile(file)
    setImagePreview(URL.createObjectURL(file))
    setError('')
  }

  const handleUploadNewImage = async () => {
    if (!imageFile) return
    try {
      setUploading(true)
                                const     res     =     await
uploadService.uploadImage(imageFile)
      if (res.success && res.data?.url) {
        setCurrentImageUrl(res.data.url)
        setImageFile(null)
        setImagePreview(null)
        setSuccessMsg('New image uploaded successfully.')
        setTimeout(() => setSuccessMsg(''), 3000)
      }
    } catch (err) {
          console.warn('Image  upload  failed,  using  local
blob:', err)
      setCurrentImageUrl(imagePreview)
      setImageFile(null)
      setImagePreview(null)
    } finally {
      setUploading(false)
    }
```

```
  }

  // Specs & Steps
  const handleSpecChange = (index, field, value) => {
    setSpecs((prev) => {
      const copy = [...prev]
      copy[index] = { ...copy[index], [field]: value }
      return copy
    })
  }

   const addSpec = () => setSpecs((prev) => [...prev, {
name: '', quantity: '' }])
  const removeSpec = (index) => {
    if (specs.length <= 1) return
    setSpecs((prev) => prev.filter((_, i) => i !== index))
  }

  const handleStepChange = (index, value) => {
    setSteps((prev) => {
      const copy = [...prev]
      copy[index] = value
      return copy
    })
  }

  const addStep = () => setSteps((prev) => [...prev, ''])
  const removeStep = (index) => {
    if (steps.length <= 1) return
    setSteps((prev) => prev.filter((_, i) => i !== index))
  }

  // Submit update
  const handleSubmit = async (e) => {
    e.preventDefault()
    setError('')

                        if      (!formData.name.trim()      ||
formData.name.trim().length < 3) {
            setError('Product  name  must  be  at  least  3
characters.')
      return
    }

    const cleanSpecs = specs
      .filter((s) => s.name.trim())
      .map((s) => ({
        name: s.name.trim(),
        quantity: s.quantity.trim() || 'Standard',
      }))

    if (cleanSpecs.length === 0) {
               setError('Please   provide   at   least   one
specification.')
      return
    }
```

```
    const cleanSteps = steps
      .filter((s) => s.trim())
      .map((s) => s.trim())

    if (cleanSteps.length === 0) {
        setError('Please provide at least one maintenance
step.')
      return
    }

    const payload = {
      name: formData.name.trim(),
          type: formData.type || (formData.category ===
'minuman' ? 'Rifle' : 'Pistol'),
      caliber: formData.caliber || '9mm',
      price: Number(formData.price) || 500,
      category: formData.category || 'makanan',
             description: formData.description.trim() ||
`${formData.type} caliber ${formData.caliber}.`,
          image_url: imagePreview || currentImageUrl ||
'/guns/pistol.svg',
      prep_time: 15,
      cook_time: Number(formData.price) || 500,
      servings: Number(formData.servings) || 1,
      difficulty: formData.difficulty || 'mudah',
      is_featured: Boolean(formData.is_featured),
      ingredients: cleanSpecs,
      steps: cleanSteps,
    }

    try {
      setUpdating(true)
         const res = await gunService.updateGun(targetId,
payload)
      if (res.success || res.data) {
        deleteDraft(targetId)
        if (onSuccess) {
          onSuccess(res.data)
        } else if (onBack) {
          onBack()
        }
      } else {
          throw new Error(res.message || 'Failed to update
record on server.')
      }
    } catch (err) {
          setError(err.message || 'Error occurred while
updating record.')
    } finally {
      setUpdating(false)
    }
  }

  if (loading) {
    return (
```

```
      <div className="state-box">
        <Loader size={32} className="spin" />
        <p>Loading hardware details for editing…</p>
      </div>
    )
  }

  return (
    <div className="form-page">
      {/* Head */}
      <div className="form-head">
             <button  type="button"  className="detail-back"
onClick={onBack}>
          <ArrowLeft size={18} /> Back
        </button>
        <button
          type="button"
          className="btn btn--outline btn--sm"
          onClick={handleManualSaveDraft}
          title="Save draft"
        >
          <Save size={14} /> Save Draft
        </button>
      </div>

      <div className="form-title-group">
                 <h1  className="display">Modify  Hardware
Record</h1>
        <p className="lede">
                              Updating   record   ID:   <code
className="mono-badge">{targetId}</code>
        </p>
      </div>

      {error && (
        <div className="toast toast--error">
          <AlertCircle size={18} /> {error}
        </div>
      )}
      {successMsg && (
        <div className="toast toast--success">
          <CheckCircle size={18} /> {successMsg}
        </div>
      )}

                            <form    onSubmit={handleSubmit}
className="entry-form">
        {/* Imagery Section */}
        <div className="card-box">
          <h2 className="card-box-title">Visual Asset</h2>
          <div className="edit-img-wrap">
            <img
              src={imagePreview || currentImageUrl}
              alt="Current hardware"
              className="edit-current-img"
            />
```

```
            <div className="edit-img-controls">
              <input
                id="edit-gun-image"
                type="file"

accept="image/jpeg,image/jpg,image/png,image/webp"
                onChange={handleImageChange}
                className="hidden-file-input"
              />
                              <label  htmlFor="edit-gun-image"
className="btn btn--outline btn--sm">
                <Upload size={14} /> Select New Image File
              </label>

              {imageFile && (
                <button
                  type="button"
                  className="btn btn--primary btn--sm"
                  onClick={handleUploadNewImage}
                  disabled={uploading}
                >
                  {uploading ? (
                    <>
                         <Loader size={14} className="spin"
/> Uploading…
                    </>
                  ) : (
                    'Upload to MinIO'
                  )}
                </button>
              )}
            </div>
          </div>
        </div>

        {/* Core fields */}
        <div className="form-grid">
          <div className="form-group">
                                  <label   htmlFor="edit-name"
className="form-label">
                                    Model   Designation   <span
className="req">*</span>
            </label>
            <input
              id="edit-name"
              type="text"
              name="name"
              value={formData.name}
               onChange={(e) => setFormData((p) => ({ ...p,
name: e.target.value }))}
              className="form-input"
              required
            />
          </div>

          <div className="form-group">
```

```
                                <label   htmlFor="edit-type"
className="form-label">
                               Platform   /   Category   <span
className="req">*</span>
            </label>
            <select
              id="edit-type"
              name="type"
              value={formData.type}
              onChange={(e) =>
                setFormData((p) => ({
                  ...p,
                  type: e.target.value,
                    category: e.target.value === 'Pistol' ?
'makanan' : 'minuman',
                  }))
              }
              className="form-select"
            >
                              <option  value="Pistol">Pistol
(Sidearm)</option>
                           <option  value="Rifle">Rifle  (Long
Gun)</option>
                             <option  value="Shotgun">Shotgun
(Smoothbore)</option>
                    <option value="Submachine Gun">Submachine
Gun</option>
            </select>
          </div>
        </div>

        <div className="form-grid form-grid--3">
          <div className="form-group">
                               <label  htmlFor="edit-caliber"
className="form-label">
                                Caliber   /   Chambering   <span
className="req">*</span>
            </label>
            <input
              id="edit-caliber"
              type="text"
              name="caliber"
              value={formData.caliber}
               onChange={(e) => setFormData((p) => ({ ...p,
caliber: e.target.value }))}
              className="form-input"
              required
            />
          </div>

          <div className="form-group">
                                <label  htmlFor="edit-price"
className="form-label">
              Price (USD) <span className="req">*</span>
            </label>
            <input
```

```
              id="edit-price"
              type="number"
              name="price"
              value={formData.price}
               onChange={(e) => setFormData((p) => ({ ...p,
price: e.target.value }))}
              className="form-input"
              min="0"
              required
            />
          </div>

          <div className="form-group">
                          <label  htmlFor="edit-difficulty"
className="form-label">
                             Grade  /  Build  Quality  <span
className="req">*</span>
            </label>
            <select
              id="edit-difficulty"
              name="difficulty"
              value={formData.difficulty}
               onChange={(e) => setFormData((p) => ({ ...p,
difficulty: e.target.value }))}
              className="form-select"
            >
                           <option  value="mudah">Commercial  /
Standard</option>
                  <option value="sedang">Mil-Spec / Service
Issue</option>
                        <option  value="sulit">Match  Grade  /
Custom</option>
            </select>
          </div>
        </div>

        <div className="form-group">
                                <label   htmlFor="edit-desc"
className="form-label">
            Armory Description
          </label>
          <textarea
            id="edit-desc"
            name="description"
            value={formData.description}
              onChange={(e) => setFormData((p) => ({ ...p,
description: e.target.value }))}
            rows={3}
            className="form-textarea"
          />
        </div>

        {/* Specs */}
        <div className="form-fieldset">
          <div className="fieldset-header">
```

```
              <label className="form-label">Specifications
&amp;amp; Components</label>
                    <button  type="button"  className="btn
btn--outline btn--sm" onClick={addSpec}>
              <Plus size={14} /> Add Spec
            </button>
          </div>
          <div className="multi-rows">
            {specs.map((spec, i) => (
              <div key={i} className="multi-row">
                <input
                  type="text"
                  placeholder="Attribute"
                  value={spec.name}
                       onChange={(e) => handleSpecChange(i,
'name', e.target.value)}
                  className="form-input"
                />
                <input
                  type="text"
                  placeholder="Value"
                  value={spec.quantity}
                       onChange={(e) => handleSpecChange(i,
'quantity', e.target.value)}
                  className="form-input"
                />
                <button
                  type="button"
                  className="btn btn--danger-ghost"
                  onClick={() => removeSpec(i)}
                  disabled={specs.length <= 1}
                >
                  <X size={16} />
                </button>
              </div>
            ))}
          </div>
        </div>

        {/* Steps */}
        <div className="form-fieldset">
          <div className="fieldset-header">
               <label className="form-label">Assembly &amp;amp;
Maintenance Protocol</label>
                    <button  type="button"  className="btn
btn--outline btn--sm" onClick={addStep}>
              <Plus size={14} /> Add Step
            </button>
          </div>
          <div className="multi-rows">
            {steps.map((st, i) => (
              <div key={i} className="step-row">
                <span className="step-num">{i + 1}</span>
                <input
                  type="text"
                  value={st}
```

```
                        onChange={(e) => handleStepChange(i,
e.target.value)}
                    className="form-input"
                  />
                  <button
                    type="button"
                    className="btn btn--danger-ghost"
                    onClick={() => removeStep(i)}
                    disabled={steps.length <= 1}
                  >
                    <X size={16} />
                  </button>
                </div>
              ))}
            </div>
          </div>

          {/* Featured */}
          <div className="checkbox-row">
            <input
              id="edit-is-featured"
              type="checkbox"
              name="is_featured"
              checked={formData.is_featured}
                onChange={(e) => setFormData((p) => ({ ...p,
is_featured: e.target.checked }))}
              className="form-checkbox"
            />
                          <label  htmlFor="edit-is-featured"
className="checkbox-label">
              Feature in Armory Showcase
            </label>
          </div>

          {/* Submit */}
          <div className="form-actions-bar">
                        <button  type="button"  className="btn
btn--outline" onClick={onBack} disabled={updating}>
              Cancel
            </button>
                        <button  type="submit"  className="btn
btn--primary" disabled={updating}>
              {updating ? (
                <>
                        <Loader size={18} className="spin" />
Updating Record…
                </>
              ) : (
                'Save Changes'
              )}
            </button>
          </div>
        </form>

        {/* Restore draft dialog */}
        <ConfirmModal
```

```
        open={showDraftModal}
        title="Resume Unsaved Changes?"
          message={`An unsaved draft for this product was
saved           on           ${draftTimestamp           ?
formatDraftTime(draftTimestamp)  :  'earlier'}.  Would  you
like to restore it?`}
        confirmLabel="Restore Draft"
        cancelLabel="Discard"
        onConfirm={handleLoadDraft}
        onCancel={handleDiscardDraft}
      />
    </div>
  )
}
```

src\pages\Catalog.jsx

```
import { useState, useMemo } from 'react'
import { RefreshCw, Plus, Loader, AlertCircle, PackageOpen
} from 'lucide-react'
import { useGuns } from '../hooks/useGuns'
import GUNS_LOCAL from '../data/guns'
import GunCard from '../components/gun/GunCard'
import GunDetail from '../components/gun/GunDetail'
import                  AdvancedFilter                  from
'../components/common/AdvancedFilter'
import                   ConfirmModal                   from
'../components/modals/ConfirmModal'
import gunService from '../services/gunService'
import {
  matchesSearch,
  matchesCategory,
  sortItems,
  extractCategories,
} from '../utils/helpers'

/**
 * Catalog page — the main product listing.
 *
 * Data source priority:
 *   1. Live API via useGuns()
 *    2. Local static array (GUNS_LOCAL) when the API is
unreachable
 *
 * Features: search, category filter, sort, CRUD actions,
detail modal.
 */
export default function Catalog({ onNavigate }) {
  // ---- API data ----
    const  { guns:  apiGuns,  loading,  error,  refetch  }  =
useGuns({ limit: 50 })
  const [useLocal, setUseLocal] = useState(false)
```

```
  // Decide data source
  const rawGuns = useMemo(() => {
    if (useLocal || (error && apiGuns.length === 0)) {
          // Enrich local data with synthetic ids so
FavoriteButton and detail work
      return GUNS_LOCAL.map((g, i) => ({ ...g, id: g.id ||
`local-${i}` }))
    }
    return apiGuns
  }, [apiGuns, error, useLocal])

  // ---- filters / sort ----
  const [search, setSearch] = useState('')
      const [activeCategory, setActiveCategory] =
useState('All')
  const [sortKey, setSortKey] = useState('name')
  const [sortOrder, setSortOrder] = useState('asc')

          const categories = useMemo(() =>
extractCategories(rawGuns), [rawGuns])

  const filteredGuns = useMemo(() => {
    let result = rawGuns.filter(
                (g) => matchesSearch(g, search) &&
matchesCategory(g, activeCategory),
    )
    return sortItems(result, sortKey, sortOrder)
      }, [rawGuns, search, activeCategory, sortKey,
sortOrder])

  // ---- detail view ----
  const [selectedGun, setSelectedGun] = useState(null)

  // ---- delete ----
  const [deleteTarget, setDeleteTarget] = useState(null)
  const [deleting, setDeleting] = useState(false)
  const [deleteError, setDeleteError] = useState('')

  const handleDelete = async () => {
    if (!deleteTarget) return
    try {
      setDeleting(true)
      setDeleteError('')
      await gunService.deleteGun(deleteTarget.id)
      setDeleteTarget(null)
      refetch()
    } catch (err) {
      setDeleteError(err.message || 'Failed to delete.')
    } finally {
      setDeleting(false)
    }
  }

  // ---- If viewing a single product detail ----
  if (selectedGun) {
    return (
```

```
      <GunDetail
        gun={selectedGun}
        onBack={() => setSelectedGun(null)}
           onEdit={(g) => onNavigate && onNavigate('Edit',
g)}
                onDelete={(g) => { setSelectedGun(null);
setDeleteTarget(g) }}
      />
    )
  }

  // ---- Main catalog view ----
  return (
    <>
      <section className="masthead">
            <h1 className="display">Hardware, by the spec
sheet.</h1>
        <p className="lede">
           A small armory of pistols, rifles, and shotguns.
Every piece listed
             with its type, caliber, and price — nothing
else.
        </p>
      </section>

      <section>
        <div className="list-head">
          <h2>Current stock</h2>
          <div className="list-head-right">
            <span className="count">
                                        {filteredGuns.length}
piece{filteredGuns.length !== 1 ? 's' : ''}
            </span>
            {onNavigate && (
              <button
                type="button"
                className="btn btn--primary btn--sm"
                onClick={() => onNavigate('Create')}
              >
                <Plus size={16} /> Add
              </button>
            )}
          </div>
        </div>

        {/* Filters */}
        <AdvancedFilter
          search={search}
          onSearch={setSearch}
          categories={categories}
          activeCategory={activeCategory}
          onCategory={setActiveCategory}
          sortKey={sortKey}
          sortOrder={sortOrder}
            onSort={({ key, order }) => { setSortKey(key);
setSortOrder(order) }}
```

```
        />

        {/* States */}
        {loading && (
          <div className="state-box">
            <Loader size={28} className="spin" />
            <p>Loading products…</p>
          </div>
        )}

        {!loading && error && !useLocal && (
          <div className="state-box state-box--error">
            <AlertCircle size={28} />
            <p>{error}</p>
            <div className="state-box__actions">
                        <button type="button" className="btn
btn--outline" onClick={refetch}>
                <RefreshCw size={16} /> Retry
              </button>
              <button
                type="button"
                className="btn btn--outline"
                onClick={() => setUseLocal(true)}
              >
                View Local Catalog
              </button>
            </div>
          </div>
        )}

          {!loading && filteredGuns.length === 0 && !error
&& (
          <div className="state-box">
            <PackageOpen size={28} />
            <p>No products match your search.</p>
          </div>
        )}

        {/* Product grid */}
        {filteredGuns.length > 0 && (
          <ul className="stock">
            {filteredGuns.map((gun) => (
              <GunCard
                key={gun.id || gun.name}
                gun={gun}
                onSelect={setSelectedGun}
                                onEdit={onNavigate ? (g) =>
onNavigate('Edit', g) : undefined}
                onDelete={setDeleteTarget}
              />
            ))}
          </ul>
        )}

        {useLocal && (
          <p className="local-notice">
```

```
              Showing local catalog (server unavailable).{'
'}
            <button
              type="button"
              className="link-btn"
                     onClick={() => { setUseLocal(false);
refetch() }}
            >
              Retry server
            </button>
          </p>
        )}
      </section>

      {/* Delete confirmation */}
      <ConfirmModal
        open={!!deleteTarget}
        title="Delete Product"
              message={`Are you sure you want to delete
"${deleteTarget?.name}"? This action cannot be undone.`}
        confirmLabel="Delete"
        danger
        loading={deleting}
        onConfirm={handleDelete}
                onCancel={() => { setDeleteTarget(null);
setDeleteError('') }}
      />
      {deleteError && (
                                      <div    className="toast
toast--error">{deleteError}</div>
      )}
    </>
  )
}
```

src\pages\About.jsx

```
function About() {
  return (
    <div className="page">
      <h1 className="display">A one-room armory.</h1>
      <p className="lede">
          Bore &amp;amp; Barrel sells a short, honest list of
firearms. Each piece is priced
        from the bench — no markup theatre, no filler.
      </p>
    </div>
  )
}

export default About
```

src\pages\Contact.jsx

```
function Contact() {
  return (
    <div className="page">
      <h1 className="display">Come by the shop.</h1>
      <p className="lede">
         123 Range Road. Open Tuesday to Saturday, 10 to 6.
Call (555) 010-0100.
      </p>
    </div>
  )
}

export default Contact
```

src\pages\FavoritesPage.jsx

```
import { useMemo } from 'react'
import { Heart, Loader, AlertCircle, ArrowRight } from
'lucide-react'
import { useFavorites } from '../hooks/useFavorites'
import { useGuns } from '../hooks/useGuns'
import GUNS_LOCAL from '../data/guns'
import GunCard from '../components/gun/GunCard'

/**
 * FavoritesPage — Displays user's shortlisted / favorited
weapons.
   *  Supports  dual-mode  (server  API  +  localStorage
fallback).
 */
export  default  function  FavoritesPage({  onSelectGun,
onNavigateCatalog }) {
  const { favorites, loading: favLoading, error: favError,
refetch } = useFavorites()
  const { guns: apiGuns } = useGuns({ limit: 100 })

  // All known guns catalog for fallback item lookup
  const allKnownGuns = useMemo(() => {
    const map = new Map()
    // Local catalog
    GUNS_LOCAL.forEach((g, idx) => {
      const id = g.id || `local-${idx}`
      const enriched = { ...g, id }
      map.set(id, enriched)
      map.set(String(id), enriched)
      if (g.name) map.set(g.name, enriched)
    })
    // API catalog
    apiGuns.forEach((g) => {
      if (g.id !== undefined && g.id !== null) {
        map.set(g.id, g)
        map.set(String(g.id), g)
      }
      if (g.name) map.set(g.name, g)
```

```
    })
    return map
  }, [apiGuns])

  // Resolve favorited products
  const favoriteItems = useMemo(() => {
     // 1. If backend returned full objects in favorites
array:
    const resolved = []
    const seen = new Set()

    favorites.forEach((fav) => {
      const id = fav.id || fav.recipe_id || fav.gun_id
                const   match   =   allKnownGuns.get(id)   ||
allKnownGuns.get(fav.name) || (fav.name ? fav : null)
      if (match && !seen.has(match.id || match.name)) {
        seen.add(match.id || match.name)
        resolved.push(match)
      } else if (id && !seen.has(id)) {
        seen.add(id)
        resolved.push({
          id,
          name: fav.name || 'Hardware item',
          type: fav.category || 'Hardware',
          image_url: fav.image_url || '/guns/pistol.svg',
          price: fav.cook_time || 0,
        })
      }
    })

    return resolved
  }, [favorites, allKnownGuns])

  const isLoading = favLoading && favoriteItems.length ===
0

  return (
    <div className="fav-page">
      <section className="masthead">
        <h1 className="display">Saved Shortlist</h1>
        <p className="lede">
            Weapons and equipment marked for procurement.
Persisted with your armory profile.
        </p>
      </section>

      <section className="fav-content">
        <div className="list-head">
          <h2>Armory Favorites</h2>
          <span className="count">
                                    {favoriteItems.length}
weapon{favoriteItems.length !== 1 ? 's' : ''} saved
          </span>
        </div>

        {isLoading && (
```

```
          <div className="state-box">
            <Loader size={28} className="spin" />
            <p>Loading your shortlist…</p>
          </div>
        )}

           {!isLoading && favError && favoriteItems.length
=== 0 && (
          <div className="state-box state-box--error">
            <AlertCircle size={28} />
            <p>{favError}</p>
                      <button  type="button"  className="btn
btn--outline" onClick={refetch}>
              Retry Connection
            </button>
          </div>
        )}

            {!isLoading && favoriteItems.length === 0 &&
!favError && (
          <div className="state-box">
            <Heart size={36} stroke="var(--steel)" />
            <h3>No hardware favorited yet</h3>
            <p className="lede">
                 Explore the armory catalog and click the
heart icon on any piece to save it here.
            </p>
            {onNavigateCatalog && (
              <button
                type="button"
                className="btn btn--primary"
                onClick={onNavigateCatalog}
                style={{ marginTop: '16px' }}
              >
                Browse Catalog <ArrowRight size={16} />
              </button>
            )}
          </div>
        )}

        {favoriteItems.length > 0 && (
          <ul className="stock">
            {favoriteItems.map((gun) => (
              <GunCard
                key={gun.id || gun.name}
                gun={gun}
                onSelect={onSelectGun}
              />
            ))}
          </ul>
        )}
      </section>
    </div>
  )
}
```

src\pages\ProfilePage.jsx

```
import { useState } from 'react'
import {
  Shield,
  Copy,
  Check,
  Save,
  Heart,
  FileText,
  Camera,
  Play,
} from 'lucide-react'
import userService from '../services/userService'
import { useFavorites } from '../hooks/useFavorites'

/**
 * Get count of active drafts in localStorage
 */
const getActiveDraftCount = () => {
  try {
    let count = 0
    for (let i = 0; i < localStorage.length; i++) {
      const key = localStorage.key(i)
       if (key && (key.startsWith('bore_barrel_draft_') ||
key.startsWith('recipe_draft_'))) {
        count++
      }
    }
    return count
  } catch {
    return 0
  }
}

/**
 * ProfilePage — User identity and Armory settings.
 * Manages operator profile, identifier, favorites stats,
and draft storage.
 */
export default function ProfilePage({ onReplaySplash }) {
      const   [profile,   setProfile]   =   useState(()   =>
userService.getUserProfile())
            const      [username,      setUsername]      =
useState(profile.username || 'Shooter')
  const [bio, setBio] = useState(profile.bio || '')
   const  [avatar,  setAvatar]  =  useState(profile.avatar  ||
null)
  const [copied, setCopied] = useState(false)
  const [feedback, setFeedback] = useState('')

  const { favorites } = useFavorites()
  const [draftCount] = useState(getActiveDraftCount)
```

```
  const handleCopyId = () => {
    if (profile.userId) {
      navigator.clipboard.writeText(profile.userId)
      setCopied(true)
      setTimeout(() => setCopied(false), 2000)
    }
  }

  const handleAvatarChange = (e) => {
    const file = e.target.files[0]
    if (!file) return

    if (file.size > 2 * 1024 * 1024) {
      setFeedback('Avatar image must be under 2MB.')
      setTimeout(() => setFeedback(''), 3000)
      return
    }

    const reader = new FileReader()
    reader.onload = (event) => {
      const base64 = event.target.result
      setAvatar(base64)
      userService.updateAvatar(base64)
      setFeedback('Avatar updated.')
      setTimeout(() => setFeedback(''), 3000)
    }
    reader.readAsDataURL(file)
  }

  const handleSaveProfile = (e) => {
    e.preventDefault()
    const updated = {
      ...profile,
      username: username.trim() || 'Shooter',
      bio: bio.trim(),
      avatar,
    }
    const result = userService.saveUserProfile(updated)
    if (result.success) {
      setProfile(result.data)
      setFeedback('Profile successfully updated.')
      setTimeout(() => setFeedback(''), 3000)
    } else {
      setFeedback('Failed to save profile.')
    }
  }

  return (
    <div className="profile-page">
          <section  className="masthead"  style={{  display:
'flex',   justifyContent:   'space-between',   alignItems:
'flex-start', flexWrap: 'wrap', gap: '12px' }}>
        <div>
          <h1 className="display">Operator Profile</h1>
          <p className="lede">
```

```
              Operator credentials, procurement statistics,
and armorer details.
          </p>
        </div>
        {onReplaySplash && (
          <button
            type="button"
            className="btn btn--outline btn--sm"
            onClick={onReplaySplash}
            title="Replay introductory splash animation"
          >
            <Play size={14} /> Replay Intro
          </button>
        )}
      </section>

      {feedback && (
        <div className="toast toast--success">
          <Check size={18} /> {feedback}
        </div>
      )}

      {/* Operator Card */}
      <div className="profile-card">
        <div className="profile-card-top">
          <div className="avatar-wrap">
            {avatar ? (
                  <img src={avatar} alt={profile.username}
className="avatar-img" />
            ) : (
              <div className="avatar-placeholder">
                <Shield size={36} stroke="var(--brass)" />
              </div>
            )}
                        <label  htmlFor="avatar-file-input"
className="avatar-upload-btn" title="Upload avatar">
              <Camera size={14} />
              <input
                id="avatar-file-input"
                type="file"
                accept="image/*"
                onChange={handleAvatarChange}
                className="hidden-file-input"
              />
            </label>
          </div>

          <div className="profile-identity">
               <h2 className="display">{profile.username ||
'Shooter'}</h2>
            <div className="profile-id-row">
                                                    <span
className="mono-badge">{profile.userId}</span>
              <button
                type="button"
                className="icon-btn"
```

```
                onClick={handleCopyId}
                title="Copy User ID to clipboard"
              >
                                {copied ? <Check size={14}
color="var(--brass)" /> : <Copy size={14} />}
              </button>
            </div>
          </div>
        </div>

        {/* Stats Grid */}
        <div className="profile-stats">
          <div className="stat-item">
                    <Heart size={20} className="stat-icon"
stroke="var(--brass)" />
                                                        <div
className="stat-val">{favorites.length}</div>
                    <div className="stat-label">Procurement
Favorites</div>
          </div>
          <div className="stat-item">
                  <FileText size={20} className="stat-icon"
stroke="var(--steel)" />
            <div className="stat-val">{draftCount}</div>
                 <div className="stat-label">Saved Offline
Drafts</div>
          </div>
        </div>
      </div>

      {/* Profile Form */}
                      <form   onSubmit={handleSaveProfile}
className="profile-form">
        <h3 className="section-title">Armorer Details</h3>

        <div className="form-group">
                                <label   htmlFor="prof-name"
className="form-label">
            Callsign / Operator Alias
          </label>
          <input
            id="prof-name"
            type="text"
            value={username}
            onChange={(e) => setUsername(e.target.value)}
                 placeholder="e.g. Operator Delta, Master
Gunsmith"
            className="form-input"
          />
        </div>

        <div className="form-group">
                                 <label   htmlFor="prof-bio"
className="form-label">
            Armory Notes / Station Bio
          </label>
```

```
          <textarea
            id="prof-bio"
            value={bio}
            onChange={(e) => setBio(e.target.value)}
                    placeholder="Qualifications,  preferred
calibers, unit assignment..."
            rows={3}
            className="form-textarea"
          />
        </div>

                    <button   type="submit"   className="btn
btn--primary">
          <Save size={16} /> Save Armorer Profile
        </button>
      </form>
    </div>
  )
}
```

6. Lanjut ke *folder* components, edit tiap *file* yang ada dengan kode ini

src/components/common/AdvancedFilter.jsx

```
import { Search, SlidersHorizontal } from 'lucide-react'

/**
 * Search bar + category pills + sort dropdown.
 *
 * @param {Object}   props
 * @param {string}   props.search        — current search
query
 * @param {Function} props.onSearch      — (value) => void
 * @param {string[]} props.categories    — e.g.
['All','Pistol','Rifle','Shotgun']
 * @param {string}   props.activeCategory
 * @param {Function} props.onCategory    — (value) => void
 * @param {string}   props.sortKey       — 'name' |
'price'
 * @param {string}   props.sortOrder     — 'asc' | 'desc'
 * @param {Function} props.onSort        — ({ key, order
}) => void
 */
export default function AdvancedFilter({
  search,
  onSearch,
  categories,
  activeCategory,
  onCategory,
  sortKey,
  sortOrder,
  onSort,
}) {
```

```
  return (
    <div className="adv-filter">
      {/* Search */}
      <div className="adv-filter__search">
        <Search size={16} className="adv-filter__icon" />
        <input
          type="search"
          placeholder="Search products…"
          value={search}
          onChange={(e) => onSearch(e.target.value)}
          className="adv-filter__input"
          aria-label="Search products"
        />
      </div>

      {/* Category pills */}
      <div className="adv-filter__pills">
        {categories.map((cat) => (
          <button
            key={cat}
            type="button"
            className={`pill${activeCategory === cat ? '
pill--active' : ''}`}
            onClick={() => onCategory(cat)}
          >
            {cat}
          </button>
        ))}
      </div>

      {/* Sort */}
      <div className="adv-filter__sort">
        <SlidersHorizontal size={14} />
        <select
          value={`${sortKey}_${sortOrder}`}
          onChange={(e) => {
            const [key, order] = e.target.value.split('_')
            onSort({ key, order })
          }}
          aria-label="Sort products"
          className="adv-filter__select"
        >
          <option value="name_asc">Name A → Z</option>
          <option value="name_desc">Name Z → A</option>
          <option value="price_asc">Price Low →
High</option>
          <option value="price_desc">Price High →
Low</option>
        </select>
      </div>
    </div>
  )
}
```

src/components/common/FavoriteButton.jsx

```
import { Heart } from 'lucide-react'
import { useIsFavorited } from '../../hooks/useFavorites'

/**
 * A toggleable heart button.  Works with both
server-backed and
 * localStorage-only favourites.
 *
 * @param {Object}   props
 * @param {string}   props.gunId  — product UUID (or local
id)
 * @param {Function} [props.onToggle] — optional callback
after toggle
 */
export default function FavoriteButton({ gunId, onToggle
}) {
  const { isFavorited, loading, toggleFavorite } =
useIsFavorited(gunId)

  const handleClick = async (e) => {
    e.stopPropagation()          // don't bubble into card
click
    const result = await toggleFavorite()
    if (result && onToggle) onToggle(result)
  }

  return (
    <button
      type="button"
      className={`fav-btn${isFavorited ? '
fav-btn--active' : ''}`}
      onClick={handleClick}
      disabled={loading}
      aria-label={isFavorited ? 'Remove from favorites' :
'Add to favorites'}
      title={isFavorited ? 'Remove from favorites' : 'Add
to favorites'}
    >
      <Heart
        size={18}
        fill={isFavorited ? 'var(--brass)' : 'none'}
        stroke={isFavorited ? 'var(--brass)' :
'var(--steel)'}
      />
    </button>
  )
}
```

src/components/common/SplashScreen.css

```
/*
============================================================
```

```
================
   Bore & Barrel — Splash Screen Animation
   Gunshop Asli Ngawi Signature Intro

============================================================
================ */

.splash-overlay {
  position: fixed;
  inset: 0;
  width: 100vw;
  height: 100vh;
  height: 100dvh;
  overflow: hidden;
  background: radial-gradient(ellipse at 50% 42%, #160e08
0%, #08090b 43%, #030406 78%);
  isolation: isolate;
  z-index: 99999;
  transition: opacity 0.6s cubic-bezier(0.16, 1, 0.3, 1),
visibility 0.6s;
  cursor: pointer;
  user-select: none;
}

.splash-overlay--fading {
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
}

.splash-mark {
  position: absolute;
  left: 50%;
  top: 29%;
  width: min(90vw, 58vh, 650px);
  aspect-ratio: 1;
  object-fit: contain;
  transform: translate(-50%, 0) translateX(-100vw)
scale(0.94);
  opacity: 0;
  animation: splash-rush 2s cubic-bezier(0.15, 0.8, 0.18,
1) 0.05s forwards,
             splash-glow 1.5s ease-out 2.55s forwards;
  mix-blend-mode: normal;
  pointer-events: none;
}

.splash-trail {
  position: absolute;
  left: -50%;
  top: 49%;
  width: 58vw;
  height: 4px;
  background: linear-gradient(90deg, transparent, #ff6500,
#ffb329, transparent);
```

```
  filter: blur(4px);
  opacity: 0;
  animation: splash-trail 1.5s ease-out 0.1s forwards;
  pointer-events: none;
}

.splash-flash {
  position: absolute;
  inset: 0;
  background: #fff0c4;
  opacity: 0;
  pointer-events: none;
  animation: splash-flash 0.23s ease-out 2.45s;
}

.splash-cracks {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  pointer-events: none;
  animation: splash-crack 0.65s ease-out 2.48s forwards;
  filter: drop-shadow(0 0 2px rgba(230, 240, 255, 0.28));
}

.splash-cracks path {
  fill: none;
  stroke: #cbd7e3;
  stroke-width: 1.2;
  stroke-linecap: round;
  stroke-linejoin: round;
  opacity: 0.82;
}

.splash-overlay.hit {
  animation: splash-shake 0.35s linear;
}

.splash-brand {
  position: absolute;
  top: 81%;
  left: 0;
  width: 100%;
  text-align: center;
  opacity: 0;
  transform: translateY(12px);
  animation: splash-brandIn 0.9s ease-out 3.8s forwards;
  color: #f6f3ec;
  pointer-events: none;
}

.splash-brand-name {
  font-family: Georgia, 'Times New Roman', serif;
  font-size: clamp(28px, 5.4vw, 43px);
```

```
  letter-spacing: 0.035em;
  font-weight: 700;
  text-shadow: 0 2px 14px rgba(0, 0, 0, 0.8);
}

.splash-brand-sub {
  margin-top: 12px;
  font-size: 11px;
  letter-spacing: 0.28em;
  color: #ff8a28;
  font-weight: 600;
  text-transform: uppercase;
}

.splash-brand::after {
  content: '';
  display: block;
  width: 76px;
  height: 1px;
  background: linear-gradient(90deg, transparent, #ff7b12,
transparent);
  margin: 16px auto 0;
}

/* Skip / Enter Action */
.splash-skip-btn {
  position: absolute;
  bottom: 24px;
  right: 24px;
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 8px 16px;
  background: rgba(18, 14, 11, 0.65);
  border: 1px solid rgba(255, 138, 40, 0.35);
  border-radius: 9999px;
  color: #f6f3ec;
  font-size: 12px;
  font-weight: 500;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  backdrop-filter: blur(8px);
  cursor: pointer;
  z-index: 10;
  opacity: 0;
  animation: splash-skipIn 0.6s ease-out 1.5s forwards;
  transition: all 0.2s ease;
}

.splash-skip-btn:hover {
  background: rgba(255, 117, 0, 0.2);
  border-color: #ff8a28;
  color: #ffffff;
  transform: translateY(-1px);
}
```

```
.splash-hint {
  position: absolute;
  bottom: 28px;
  left: 24px;
  font-size: 11px;
  color: rgba(246, 243, 236, 0.35);
  letter-spacing: 0.05em;
  opacity: 0;
  animation: splash-skipIn 0.6s ease-out 2.5s forwards;
  pointer-events: none;
}

/* Animations */
@keyframes splash-rush {
  0% {
    opacity: 0;
    left: -20%;
    transform: translate(-50%, 0) scale(0.94);
  }
  58% {
    opacity: 1;
    left: 53%;
    transform: translate(-50%, 0) scale(1.025);
  }
  78% {
    left: 49%;
    transform: translate(-50%, 0) scale(0.99);
  }
  100% {
    opacity: 1;
    left: 50%;
    transform: translate(-50%, 0) scale(1);
  }
}

@keyframes splash-trail {
  0% {
    opacity: 0;
    left: -35%;
  }
  22% {
    opacity: 0.95;
  }
  100% {
    opacity: 0;
    left: 48%;
  }
}

@keyframes splash-flash {
  0% {
    opacity: 0;
  }
  25% {
```

```
    opacity: 0.75;
  }
  100% {
    opacity: 0;
  }
}

@keyframes splash-crack {
  0% {
    opacity: 0;
    transform: scale(0.98);
  }
  45% {
    opacity: 0.9;
  }
  100% {
    opacity: 0.82;
    transform: scale(1);
  }
}

@keyframes splash-shake {
  0%, 100% {
    transform: translate(0);
  }
  20% {
    transform: translate(-5px, 2px);
  }
  45% {
    transform: translate(4px, -2px);
  }
  70% {
    transform: translate(-2px, 1px);
  }
}

@keyframes splash-glow {
  0% {
    filter: drop-shadow(0 0 0 rgba(255, 117, 0, 0));
  }
  45% {
    filter: drop-shadow(0 0 24px rgba(255, 117, 0, 0.5));
  }
  100% {
    filter: drop-shadow(0 0 10px rgba(255, 117, 0, 0.2));
  }
}

@keyframes splash-brandIn {
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```
@keyframes splash-skipIn {
  to {
    opacity: 1;
  }
}

@media (prefers-reduced-motion: reduce) {
  .splash-overlay *,
  .splash-overlay *::before,
  .splash-overlay *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
  }
}
```

src/components/common/SplashScreen.jsx

```
import { useState, useEffect, useCallback, useRef } from
'react'
import { ArrowRight } from 'lucide-react'
import './SplashScreen.css'

/**
 * SplashScreen — Full-bleed signature splash animation
for Bore & Barrel
 * (Gunshop Asli Ngawi).
 *
 * Features:
 * - Dynamic rush & glow animation on logo
 * - Bullet trail, muzzle flash, glass crack SVG
 * - Interactive recoil/shake effect on click
 * - Auto-dismiss after 5.4s with smooth fade-out
 * - Skip / Enter Armory button
 *
 * @param {Object} props
 * @param {Function} [props.onFinish] - Callback when
splash finishes / is dismissed
 * @param {number} [props.duration=5400] - Duration in ms
before auto-fading
 */
export default function SplashScreen({ onFinish, duration
= 5400 }) {
  const [isHit, setIsHit] = useState(false)
  const [isFading, setIsFading] = useState(false)
  const dismissedRef = useRef(false)

  const handleDismiss = useCallback(() => {
    if (dismissedRef.current) return
    dismissedRef.current = true
    setIsFading(true)
    setTimeout(() => {
      if (onFinish) onFinish()
```

```
    }, 600)
  }, [onFinish])

  // Interactive recoil on tap/click
  const handleHit = (e) => {
    // If user clicked the skip button, let that handler
run instead
    if (e.target.closest('.splash-skip-btn')) return

    setIsHit(false)
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        setIsHit(true)
      })
    })
  }

  // Auto-dismiss timer
  useEffect(() => {
    const timer = setTimeout(() => {
      handleDismiss()
    }, duration)

    const handleKeyDown = (e) => {
      if (e.key === 'Escape' || e.key === ' ' || e.key ===
'Enter') {
        handleDismiss()
      }
    }

    window.addEventListener('keydown', handleKeyDown)

    return () => {
      clearTimeout(timer)
      window.removeEventListener('keydown', handleKeyDown)
    }
  }, [duration, handleDismiss])

  return (
    <main
      className={`splash-overlay ${isHit ? 'hit' : ''}
${isFading ? 'splash-overlay--fading' : ''}`}
      id="splash"
      onClick={handleHit}
      role="dialog"
      aria-modal="true"
      aria-label="Bore & Barrel Intro"
    >
      {/* Light bullet trail */}
      <div className="splash-trail" aria-hidden="true" />

      {/* Muzzle flash */}
      <div className="splash-flash" aria-hidden="true" />

      {/* Gunshop Asli Ngawi cropped emblem */}
```

```
      <img
        className="splash-mark"
        src="/logo.png"
        alt="Bore & Barrel Gunshop Asli Ngawi Logo"
      />

      {/* Bullet impact cracks SVG */}
      <svg
        className="splash-cracks"
        viewBox="0 0 720 1280"
        preserveAspectRatio="none"
        aria-hidden="true"
      >
        <g transform="translate(525 500)">
          <path d="M0 0 28 -24 61 -31 84 -58 120 -72 158
-91 196 -128 230 -145 M61 -31 74 5 108 26 125 58 M120 -72
134 -42 166 -23 184 14 M0 0 -34 -22 -72 -27 -95 -52 -128
-61 -159 -92 M-72 -27 -69 7 -92 31 -105 63 M-128 -61 -148
-40 -176 -38 -204 -17 M0 0 14 38 10 75 35 103 42 137 M14
38 48 49 71 76 96 88 M0 0 -31 27 -55 60 -83 81 -105 119
M-55 60 -40 91 -49 119 M0 0 40 13 68 8 96 29 129 32 M40 13
44 -18 66 -39 M0 0 7 -45 -2 -76 13 -108 5 -139 M7 -45 33
-64 53 -91" />
          <path d="M-159 -92 -173 -119 -198 -133 M196 -128
214 -101 245 -88 M42 137 66 163 70 194 M-105 119 -128 144
-154 156 M230 -145 254 -164 279 -166" />
        </g>
      </svg>

      {/* Brand title reveal */}
      <section className="splash-brand">
        <div className="splash-brand-name">Bore &amp;
Barrel</div>
        <div className="splash-brand-sub">GUNSHOP ASLI
NGAWI</div>
      </section>

      {/* Skip button */}
      <button
        type="button"
        className="splash-skip-btn"
        onClick={(e) => {
          e.stopPropagation()
          handleDismiss()
        }}
        aria-label="Skip splash screen"
      >
        <span>Enter Armory</span>
        <ArrowRight size={14} />
      </button>

      <span className="splash-hint">Tap anywhere to
recoil</span>
    </main>
  )
```

```
}
```

src/components/gun/GunCard.jsx

```
import FavoriteButton from '../common/FavoriteButton'
import { formatPrice, truncate } from
'../../utils/helpers'

/**
 * Product card used in the catalog grid.
 *
 * @param {Object}   props
 * @param {Object}   props.gun       — product data
 * @param {Function} props.onSelect  — called with the gun
when clicked
 * @param {Function} [props.onEdit]  — called when Edit is
clicked (admin)
 * @param {Function} [props.onDelete]— called when Delete
is clicked (admin)
 */
export default function GunCard({ gun, onSelect, onEdit,
onDelete }) {
  const imgSrc = gun.image_url || gun.image ||
'/guns/pistol.svg'
  const category = gun.type || gun.category || '—'
  const caliber = gun.caliber || gun.difficulty || ''
  const price = gun.price ?? gun.cook_time ?? 0

  return (
    <li className="card">
      <button
        type="button"
        className="card-btn"
        onClick={() => onSelect(gun)}
        aria-label={`View ${gun.name}`}
      >
        <img
          className="card-img"
          src={imgSrc}
          alt={gun.name}
          width="240"
          height="180"
          loading="lazy"
        />

        <span className="name display">{gun.name}</span>

        <span className="type">
          {category}
          {caliber ? ` · ${caliber}` : ''}
        </span>
```

```
        {gun.description && (
          <span
className="card-desc">{truncate(gun.description,
72)}</span>
        )}

        <span
className="price">{formatPrice(price)}</span>
      </button>

      {/* Floating actions row */}
      <div className="card-actions">
        <FavoriteButton gunId={gun.id || gun.name} />

        {onEdit && (
          <button
            type="button"
            className="card-act-btn"
            onClick={(e) => { e.stopPropagation();
onEdit(gun) }}
            aria-label={`Edit ${gun.name}`}
            title="Edit"
          >
            ✏️
          </button>
        )}

        {onDelete && (
          <button
            type="button"
            className="card-act-btn card-act-btn--danger"
            onClick={(e) => { e.stopPropagation();
onDelete(gun) }}
            aria-label={`Delete ${gun.name}`}
            title="Delete"
          >
            🗑️
          </button>
        )}
      </div>
    </li>
  )
}
```

src/components/gun/GunDetail.jsx

```
import { useState } from 'react'
import { ArrowLeft, Star, Send, Loader } from
'lucide-react'
import FavoriteButton from '../common/FavoriteButton'
import { useReviews, useCreateReview } from
'../../hooks/useReviews'
```

```
import { getUserIdentifier } from
'../../services/userService'
import { formatPrice, averageRating } from
'../../utils/helpers'

/**
 * Full-screen product detail view.
 *
 * @param {Object}   props
 * @param {Object}   props.gun       — product data
 * @param {Function} props.onBack    — return to catalog
 * @param {Function} [props.onEdit]  — navigate to edit
page
 * @param {Function} [props.onDelete]— trigger delete flow
 */
export default function GunDetail({ gun, onBack, onEdit,
onDelete }) {
  const imgSrc = gun.image_url || gun.image ||
'/guns/pistol.svg'
  const category = gun.type || gun.category || '—'
  const caliber = gun.caliber || gun.difficulty || ''
  const price = gun.price ?? gun.cook_time ?? 0

  // Reviews
  const { reviews, loading: reviewsLoading, refetch:
refetchReviews } = useReviews(gun.id)
  const { createReview, loading: submitting } =
useCreateReview()
  const [rating, setRating] = useState(5)
  const [comment, setComment] = useState('')
  const [reviewError, setReviewError] = useState('')
  const [reviewSuccess, setReviewSuccess] = useState('')

  const avgRating = gun.average_rating ||
averageRating(reviews)
  const reviewCount = gun.review_count ?? reviews.length

  const handleSubmitReview = async (e) => {
    e.preventDefault()
    setReviewError('')
    setReviewSuccess('')

    if (rating < 1 || rating > 5) {
      setReviewError('Rating must be between 1 and 5.')
      return
    }

    const result = await createReview(gun.id, {
      user_identifier: getUserIdentifier(),
      rating,
      comment: comment.trim(),
    })

    if (result) {
      setReviewSuccess('Review submitted!')
```

```
      setComment('')
      setRating(5)
      refetchReviews()
    } else {
      setReviewError('Could not submit review. The server
may be unavailable.')
    }
  }

  return (
    <div className="detail">
      {/* Back button */}
      <button type="button" className="detail-back"
onClick={onBack}>
        <ArrowLeft size={18} /> Back to Catalog
      </button>

      {/* Hero */}
      <div className="detail-hero">
        <img className="detail-img" src={imgSrc}
alt={gun.name} />
        <div className="detail-info">
          <h1 className="display">{gun.name}</h1>
          <p className="type">
            {category}
            {caliber ? ` · ${caliber}` : ''}
          </p>
          <p className="price
detail-price">{formatPrice(price)}</p>

          {avgRating > 0 && (
            <p className="detail-rating">
              <Star size={16} fill="var(--brass)"
stroke="var(--brass)" />
              {avgRating.toFixed(1)} ({reviewCount}
review{reviewCount !== 1 ? 's' : ''})
            </p>
          )}

          <div className="detail-actions">
            <FavoriteButton gunId={gun.id || gun.name} />
            {onEdit && (
              <button type="button" className="btn
btn--outline" onClick={() => onEdit(gun)}>
                Edit
              </button>
            )}
            {onDelete && (
              <button type="button" className="btn
btn--danger" onClick={() => onDelete(gun)}>
                Delete
              </button>
            )}
          </div>
        </div>
```

```
      </div>

      {/* Description */}
      <section className="detail-section">
        <h2>Description</h2>
        <p>{gun.description || 'No description
available.'}</p>
      </section>

      {/* Specifications (from ingredients / steps if
present) */}
      {gun.ingredients && gun.ingredients.length > 0 && (
        <section className="detail-section">
          <h2>Specifications</h2>
          <ul className="detail-specs">
            {gun.ingredients.map((ing, i) => (
              <li key={ing.id || i}>
                <strong>{ing.name}</strong> —
{ing.quantity}
              </li>
            ))}
          </ul>
        </section>
      )}

      {gun.steps && gun.steps.length > 0 && (
        <section className="detail-section">
          <h2>Assembly / Maintenance Steps</h2>
          <ol className="detail-steps">
            {gun.steps.map((step, i) => (
              <li key={step.id || i}>
                {step.instruction || step}
              </li>
            ))}
          </ol>
        </section>
      )}

      {/* Reviews Section */}
      <section className="detail-section">
        <h2>Reviews</h2>

        {/* Submit review */}
        <form className="review-form"
onSubmit={handleSubmitReview}>
          <div className="review-stars">
            {[1, 2, 3, 4, 5].map((n) => (
              <button
                key={n}
                type="button"
                className="star-btn"
                onClick={() => setRating(n)}
                aria-label={`nstar{n > 1 ? 's' : ''}`}
              >
                <Star
```

```
                  size={22}
                  fill={n <= rating ? 'var(--brass)' :
'none'}
                  stroke={n <= rating ? 'var(--brass)' :
'var(--steel)'}
                />
              </button>
            ))}
          </div>

          <textarea
            className="review-input"
            rows="3"
            placeholder="Write a short review (optional)…"
            value={comment}
            onChange={(e) => setComment(e.target.value)}
            maxLength={500}
          />

          <button
            type="submit"
            className="btn btn--primary"
            disabled={submitting || !gun.id}
          >
            {submitting ? (
              <><Loader size={16} className="spin" />
Submitting…</>
            ) : (
              <><Send size={16} /> Submit Review</>
            )}
          </button>

          {reviewError && <p
className="form-error">{reviewError}</p>}
          {reviewSuccess && <p
className="form-success">{reviewSuccess}</p>}
          {!gun.id && (
            <p className="form-hint">
              Reviews require a server-generated product
ID. This product is local-only.
            </p>
          )}
        </form>

        {/* Existing reviews */}
        {reviewsLoading ? (
          <p className="detail-loading">Loading
reviews…</p>
        ) : reviews.length > 0 ? (
          <ul className="review-list">
            {reviews.map((rev) => (
              <li key={rev.id} className="review-item">
                <div className="review-head">
                  <span
className="review-user">{rev.user_identifier ||
```

```
'Anonymous'}</span>
                  <span className="review-stars-display">
                    {Array.from({ length: 5 }, (_, i) => (
                      <Star
                        key={i}
                        size={14}
                        fill={i < rev.rating ?
'var(--brass)' : 'none'}
                        stroke={i < rev.rating ?
'var(--brass)' : 'var(--line)'}
                      />
                    ))}
                  </span>
                </div>
                {rev.comment && <p
className="review-comment">{rev.comment}</p>}
              </li>
            ))}
          </ul>
        ) : (
          <p className="detail-empty">No reviews yet. Be
the first!</p>
        )}
      </section>
    </div>
  )
}
```

src/components/modals/ConfirmModal.jsx

```
import { AlertTriangle } from 'lucide-react'

/**
 * Accessible confirmation dialog built on the native
<dialog> element.
 *
 * @param {Object}   props
 * @param {boolean}  props.open     — whether the modal is
visible
 * @param {string}   props.title    — heading text
 * @param {string}   props.message  — body text
 * @param {string}   [props.confirmLabel='Confirm']
 * @param {string}   [props.cancelLabel='Cancel']
 * @param {boolean}  [props.danger]  — red style for
destructive actions
 * @param {boolean}  [props.loading] — disables buttons
while in progress
 * @param {Function} props.onConfirm
 * @param {Function} props.onCancel
 */
export default function ConfirmModal({
  open,
```

```
  isOpen,
  title,
  message,
  confirmLabel,
  confirmText,
  cancelLabel,
  cancelText,
  danger = false,
  loading = false,
  onConfirm,
  onCancel,
  onClose,
}) {
  const isVisible = open ?? isOpen ?? false
  const handleCancel = onCancel || onClose
  const actualConfirmText = confirmLabel || confirmText ||
'Confirm'
  const actualCancelText = cancelLabel || cancelText ||
'Cancel'

  if (!isVisible) return null

  return (
    <div className="modal-backdrop"
onClick={handleCancel}>
      <div
        className="modal-box"
        role="alertdialog"
        aria-modal="true"
        aria-labelledby="confirm-title"
        aria-describedby="confirm-msg"
        onClick={(e) => e.stopPropagation()}
      >
        <div className="modal-icon">
          <AlertTriangle size={28} stroke={danger ?
'#c0392b' : 'var(--brass)'} />
        </div>
        <h3 id="confirm-title" className="modal-title
display">
          {title}
        </h3>
        <p id="confirm-msg" className="modal-msg">
          {message}
        </p>
        <div className="modal-actions">
          <button
            type="button"
            className="modal-btn modal-btn--cancel"
            onClick={handleCancel}
            disabled={loading}
          >
            {actualCancelText}
          </button>
          <button
            type="button"
```

```
            className={`modal-btn ${danger ?
'modal-btn--danger' : 'modal-btn--confirm'}`}
            onClick={onConfirm}
            disabled={loading}
          >
            {loading ? 'Processing…' : actualConfirmText}
          </button>
        </div>
      </div>
    </div>
  )
}
```

src/components/Footer.jsx

```
function Footer() {
  return (
    <footer className="footer">
      <p>Bore &amp;amp; Barrel, 123 Range Road</p>
    </footer>
  )
}

export default Footer
```

src/components/GunCard.jsx

```
export { default } from './gun/GunCard'
```

src/components/Header.jsx

```
import { Plus } from 'lucide-react'
import { useFavorites } from '../hooks/useFavorites'

const NAV_ITEMS = [
  { id: 'Catalog', label: 'Catalog' },
  { id: 'Favorites', label: 'Favorites', hasBadge: true },
  { id: 'Profile', label: 'Profile' },
  { id: 'About', label: 'About' },
  { id: 'Contact', label: 'Contact' },
]

function Header({ tab, onTab, onNavigateCreate }) {
  const { favorites } = useFavorites()
  const favCount = favorites ? favorites.length : 0

  return (
    <header className="header">
      <div className="header-brand-wrap" onClick={() =>
```

```
onTab('Catalog')} style={{ cursor: 'pointer', display:
'flex', alignItems: 'center', gap: '10px' }}>
        <div className="header-logo-badge">
          <img src="/logo.png" alt="Bore & Barrel" />
        </div>
        <span className="brand display">Bore &amp;
Barrel</span>
      </div>

      <nav className="nav">
        {NAV_ITEMS.map((item) => (
          <button
            key={item.id}
            type="button"
            className={tab === item.id ? 'nav-link active'
: 'nav-link'}
            onClick={() => onTab(item.id)}
          >
            {item.label}
            {item.hasBadge && favCount > 0 && (
              <span
className="nav-badge">{favCount}</span>
            )}
          </button>
        ))}
      </nav>

      {onNavigateCreate && (
        <button
          type="button"
          className="btn btn--primary btn--sm header-cta"
          onClick={onNavigateCreate}
          title="Register new hardware"
        >
          <Plus size={15} /> Add Hardware
        </button>
      )}
    </header>
  )
}

export default Header
```

src/components/PWABadge.jsx

```
import { useState, useEffect } from 'react'
import { Download, WifiOff, X } from 'lucide-react'

/**
 * PWABadge — Progressive Web App installation prompt &
offline status indicator.
 */
```

```
export default function PWABadge() {
  const [installPrompt, setInstallPrompt] = useState(null)
  const [isOffline, setIsOffline] =
useState(!navigator.onLine)
  const [installed, setInstalled] = useState(false)
  const [dismissed, setDismissed] = useState(false)

  useEffect(() => {
    // Capture PWA install prompt
    const handleBeforeInstall = (e) => {
      e.preventDefault()
      setInstallPrompt(e)
    }

    const handleAppInstalled = () => {
      setInstalled(true)
      setInstallPrompt(null)
    }

    const handleOnline = () => setIsOffline(false)
    const handleOffline = () => setIsOffline(true)

    window.addEventListener('beforeinstallprompt',
handleBeforeInstall)
    window.addEventListener('appinstalled',
handleAppInstalled)
    window.addEventListener('online', handleOnline)
    window.addEventListener('offline', handleOffline)

    return () => {
      window.removeEventListener('beforeinstallprompt',
handleBeforeInstall)
      window.removeEventListener('appinstalled',
handleAppInstalled)
      window.removeEventListener('online', handleOnline)
      window.removeEventListener('offline', handleOffline)
    }
  }, [])

  const handleInstallClick = async () => {
    if (!installPrompt) return
    installPrompt.prompt()
    const { outcome } = await installPrompt.userChoice
    if (outcome === 'accepted') {
      setInstalled(true)
    }
    setInstallPrompt(null)
  }

  if (dismissed) return null

  // Offline status banner
  if (isOffline) {
    return (
      <aside className="pwa-badge pwa-badge--offline"
```

```
role="status" aria-live="polite">
        <WifiOff size={16} />
        <span>Working offline. Cached hardware data is
available.</span>
      </aside>
    )
  }

  // Install PWA banner
  if (installPrompt && !installed) {
    return (
      <aside className="pwa-badge" role="region"
aria-label="Install App">
        <div className="pwa-badge-content">
          <Download size={16} stroke="var(--brass)" />
          <span>Install <strong>Bore &amp;amp; Barrel</strong>
for offline access.</span>
        </div>
        <div className="pwa-badge-actions">
          <button
            type="button"
            className="btn btn--primary btn--sm"
            onClick={handleInstallClick}
          >
            Install App
          </button>
          <button
            type="button"
            className="icon-btn"
            onClick={() => setDismissed(true)}
            aria-label="Dismiss banner"
          >
            <X size={14} />
          </button>
        </div>
      </aside>
    )
  }

  return null
}
```

7. *Update file* berikut untuk handle operasi CRUD yang baru ditambahkan

App.jsx

```
import { useState } from 'react'
import Header from './components/Header.jsx'
import Footer from './components/Footer.jsx'
import Catalog from './pages/Catalog.jsx'
import FavoritesPage from './pages/FavoritesPage.jsx'
import ProfilePage from './pages/ProfilePage.jsx'
```

```
import CreateGunPage from './pages/CreateGunPage.jsx'
import EditGunPage from './pages/EditGunPage.jsx'
import GunDetail from './components/gun/GunDetail.jsx'
import About from './pages/About.jsx'
import Contact from './pages/Contact.jsx'
import PWABadge from './components/PWABadge.jsx'
import                  ConfirmModal                  from
'./components/modals/ConfirmModal.jsx'
import                  SplashScreen                  from
'./components/common/SplashScreen.jsx'
import gunService from './services/gunService.js'
import './App.css'

function App() {
  const [showSplash, setShowSplash] = useState(true)
  const [tab, setTab] = useState('Catalog')
   const [mode, setMode] = useState('list') // 'list' |
'create' | 'edit' | 'detail'
  const [selectedGun, setSelectedGun] = useState(null)
  const [editingGun, setEditingGun] = useState(null)
  const [deleteTarget, setDeleteTarget] = useState(null)
  const [deleting, setDeleting] = useState(false)

  // Unified navigation handler
  const handleNavigate = (target, payload) => {
    if (target === 'Create') {
      setMode('create')
      window.scrollTo({ top: 0, behavior: 'smooth' })
    } else if (target === 'Edit') {
      setEditingGun(payload)
      setMode('edit')
      window.scrollTo({ top: 0, behavior: 'smooth' })
    } else if (target === 'Detail') {
      setSelectedGun(payload)
      setMode('detail')
      window.scrollTo({ top: 0, behavior: 'smooth' })
    } else {
      // Tab change
      setTab(target)
      setMode('list')
      setSelectedGun(null)
      setEditingGun(null)
      window.scrollTo({ top: 0, behavior: 'smooth' })
    }
  }

  const handleBackToList = () => {
    setMode('list')
    setSelectedGun(null)
    setEditingGun(null)
  }

  return (
    <div className="shell">
      {showSplash && (
```

```
                          <SplashScreen   onFinish={()   =>
setShowSplash(false)} />
      )}

      <Header
        tab={mode === 'list' ? tab : ''}
        onTab={(newTab) => handleNavigate(newTab)}
        onNavigateCreate={() => handleNavigate('Create')}
      />

      <main className="main">
        {/* Sub-page views */}
        {mode === 'create' && (
          <CreateGunPage
            onBack={handleBackToList}
            onSuccess={() => {
              handleBackToList()
              setTab('Catalog')
            }}
          />
        )}

        {mode === 'edit' && (
          <EditGunPage
            gun={editingGun}
            gunId={editingGun?.id}
            onBack={handleBackToList}
            onSuccess={() => {
              handleBackToList()
              setTab('Catalog')
            }}
          />
        )}

        {mode === 'detail' && selectedGun && (
          <GunDetail
            gun={selectedGun}
            onBack={handleBackToList}
            onEdit={(g) => handleNavigate('Edit', g)}
            onDelete={(g) => setDeleteTarget(g)}
          />
        )}

        {/* Top-level Tabs */}
        {mode === 'list' && (
          <>
            {tab === 'Catalog' && (
              <Catalog
                      onNavigate={(destination, payload)  =>
handleNavigate(destination, payload)}
              />
            )}
            {tab === 'Favorites' && (
              <FavoritesPage
                                        onSelectGun={(g)   =>
handleNavigate('Detail', g)}
```

```
                                    onNavigateCatalog={()  =>
setTab('Catalog')}
                />
              )}
              {tab === 'Profile' && (
                        <ProfilePage  onReplaySplash={()  =>
setShowSplash(true)} />
              )}
              {tab === 'About' && <About />}
              {tab === 'Contact' && <Contact />}
            </>
          )}
        </main>

        {/* Global Delete Confirmation */}
        <ConfirmModal
          open={Boolean(deleteTarget)}
          title="Delete Hardware Record"
                message={`Are  you  sure  you  want  to  delete
"${deleteTarget?.name}"? This action cannot be undone.`}
          confirmLabel="Delete"
          danger
          loading={deleting}
          onConfirm={async () => {
            if (!deleteTarget) return
            try {
              setDeleting(true)
              await gunService.deleteGun(deleteTarget.id)
              setDeleteTarget(null)
              handleBackToList()
              setTab('Catalog')
            } catch (err) {
                     alert(err.message  ||  'Failed  to  delete
record.')
            } finally {
              setDeleting(false)
            }
          }}
          onCancel={() => setDeleteTarget(null)}
        />

        <PWABadge />
        <Footer />
      </div>
    )
}

export default App
```

App.css

```
/*
-------------------------------------------------------------
-----------------
```

```
 * BORE & BARREL — TACTICAL HARDWARE CATALOG & ARMORY PWA II
 * Design tokens & core typography
                                                                 *
---------------------------------------------------------------- */

.display {
   font-family: 'Arial Narrow', 'Franklin Gothic Medium', 'Roboto Condensed', system-ui,
    sans-serif;
  font-stretch: condensed;
  letter-spacing: -0.01em;
}

.shell {
  min-height: 100svh;
  display: flex;
  flex-direction: column;
  background: var(--paper);
  color: var(--ink);
}

/*
---------------------------------------------------------------- 
 * Header & Navigation
                                                                 *
---------------------------------------------------------------- */
.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 14px 24px;
  background: var(--paper);
  border-bottom: 1px solid var(--line);
  position: sticky;
  top: 0;
  z-index: 20;
}

.header-brand-wrap {
  display: flex;
  align-items: center;
  gap: 10px;
}

.header-logo-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 34px;
  height: 34px;
  background-color: #000000;
```

```
  border-radius: 8px;
  padding: 4px;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.35);
  border: 1px solid rgba(255, 255, 255, 0.12);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  flex-shrink: 0;
}

.header-brand-wrap:hover .header-logo-badge {
  transform: scale(1.06);
  box-shadow: 0 0 10px rgba(255, 123, 18, 0.45);
}

.header-logo-badge img {
  width: 100%;
  height: 100%;
  object-fit: contain;
  display: block;
}

.brand {
  font-size: 19px;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--ink);
  user-select: none;
}

.nav {
  display: flex;
  gap: 20px;
  align-items: center;
}

.nav-link {
  background: none;
  border: none;
  border-bottom: 2px solid transparent;
  padding: 6px 0;
  color: var(--steel);
  font: inherit;
  font-size: 0.95rem;
  font-weight: 500;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 4px;
  transition: color 0.15s ease, border-color 0.15s ease;
}

.nav-link:hover {
  color: var(--ink);
}

.nav-link.active {
```

```
  color: var(--ink);
  font-weight: 600;
  border-bottom-color: var(--brass);
}

.nav-badge {
  background: var(--brass);
  color: #fff;
  font-size: 0.72rem;
  font-weight: 700;
  padding: 1px 6px;
  border-radius: 999px;
  line-height: 1.2;
}

.header-cta {
  display: inline-flex;
}

@media (max-width: 640px) {
  .header {
    padding: 10px 16px;
    flex-wrap: wrap;
  }
  .nav {
    order: 3;
    width: 100%;
    overflow-x: auto;
    padding-top: 6px;
    gap: 16px;
  }
  .header-cta {
    font-size: 0.8rem;
    padding: 6px 10px;
  }
}

/*
---------------------------------------------------------------
-----------------
 * Main Container & Mastheads
                                                               *
---------------------------------------------------------------
----------------- */
.main {
  width: 100%;
  max-width: 900px;
  margin: 0 auto;
  padding: 0 24px 64px;
  flex: 1;
}

.masthead,
.page {
  padding: 48px 0 32px;
}
```

```
.masthead {
  border-bottom: 1px solid var(--line);
}

.masthead h1,
.page h1 {
  font-size: clamp(2.2rem, 5vw, 3.2rem);
  line-height: 1.05;
  letter-spacing: -0.02em;
  margin: 0 0 12px;
}

.lede {
  color: var(--steel);
  max-width: 58ch;
  margin: 0;
  font-size: 1.02rem;
  line-height: 1.5;
}

/*
----------------------------------------------------------------
-----------------
 * Reusable Buttons
                                                                *
----------------------------------------------------------------
----------------- */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  padding: 10px 18px;
  font: inherit;
  font-size: 0.92rem;
  font-weight: 600;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.15s ease;
  text-decoration: none;
  white-space: nowrap;
}

.btn:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

.btn--primary {
  background: var(--ink);
  color: #fff;
  border: 1px solid var(--ink);
}

.btn--primary:hover:not(:disabled) {
```

```
  background: #2a3a49;
  border-color: #2a3a49;
}

.btn--outline {
  background: #fff;
  color: var(--ink);
  border: 1px solid var(--line);
}

.btn--outline:hover:not(:disabled) {
  border-color: var(--steel);
  background: #fafbfb;
}

.btn--danger {
  background: #c0392b;
  color: #fff;
  border: 1px solid #c0392b;
}

.btn--danger:hover:not(:disabled) {
  background: #a93226;
  border-color: #a93226;
}

.btn--danger-ghost {
  background: transparent;
  color: #c0392b;
  border: none;
  padding: 6px 8px;
}

.btn--danger-ghost:hover:not(:disabled) {
  background: #feeceb;
  border-radius: 4px;
}

.btn--sm {
  padding: 6px 12px;
  font-size: 0.82rem;
}

.icon-btn {
  background: none;
  border: 1px solid var(--line);
  padding: 6px;
  border-radius: 4px;
  cursor: pointer;
  color: var(--steel);
  display: inline-flex;
  align-items: center;
  justify-content: center;
}

.icon-btn:hover {
```

```
  color: var(--ink);
  border-color: var(--ink);
}

/*
------------------------------------------------------------
-----------------
 * Filters & Search Bar
                                                            *
------------------------------------------------------------
----------------- */
.adv-filter {
  display: flex;
  flex-direction: column;
  gap: 14px;
  margin: 20px 0 28px;
  padding: 16px;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
}

.adv-filter__search {
  position: relative;
  width: 100%;
}

.adv-filter__icon {
  position: absolute;
  left: 12px;
  top: 50%;
  transform: translateY(-50%);
  color: var(--steel);
  pointer-events: none;
}

.adv-filter__input {
  width: 100%;
  padding: 10px 12px 10px 36px;
  border: 1px solid var(--line);
  border-radius: 4px;
  font: inherit;
  font-size: 0.95rem;
  background: var(--paper);
  color: var(--ink);
}

.adv-filter__input:focus {
  outline: none;
  border-color: var(--brass);
  background: #fff;
}

.adv-filter__pills {
  display: flex;
  flex-wrap: wrap;
```

```
  gap: 8px;
}

.pill {
  border: 1px solid var(--line);
  background: #fff;
  color: var(--steel);
  padding: 6px 14px;
  border-radius: 999px;
  font: inherit;
  font-size: 0.85rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.15s ease;
}

.pill:hover {
  border-color: var(--ink);
  color: var(--ink);
}

.pill--active {
  background: var(--ink);
  color: #fff;
  border-color: var(--ink);
  font-weight: 600;
}

.adv-filter__sort {
  display: flex;
  align-items: center;
  gap: 8px;
  color: var(--steel);
  font-size: 0.85rem;
}

.adv-filter__select {
  padding: 6px 10px;
  font: inherit;
  font-size: 0.85rem;
  border: 1px solid var(--line);
  border-radius: 4px;
  background: #fff;
  color: var(--ink);
}

/*
------------------------------------------------------------
-----------------
 * Catalog List & Product Cards
                                                            *
------------------------------------------------------------
----------------- */
.list-head {
  display: flex;
  align-items: center;
```

```
  justify-content: space-between;
  padding: 24px 0 12px;
}

.list-head h2 {
  font-size: 1.15rem;
  font-weight: 600;
  margin: 0;
}

.list-head-right {
  display: flex;
  align-items: center;
  gap: 14px;
}

.count {
  color: var(--steel);
  font-size: 0.9rem;
}

.stock {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
    grid-template-columns:  repeat(auto-fill,  minmax(240px,
1fr));
  gap: 20px;
}

.card {
  position: relative;
  display: flex;
  flex-direction: column;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  overflow: hidden;
   transition: transform 0.15s ease, box-shadow 0.15s ease,
border-color 0.15s ease;
}

.card:hover {
  border-color: var(--brass);
  box-shadow: 0 4px 14px rgba(24, 34, 44, 0.08);
}

.card-btn {
  flex: 1;
  display: flex;
  flex-direction: column;
  padding: 0 0 16px;
  background: transparent;
  border: none;
  font: inherit;
```

```
  text-align: left;
  cursor: pointer;
  color: inherit;
}

.card-img {
  width: 100%;
  height: 180px;
  object-fit: cover;
  background: #f7f9fa;
  border-bottom: 1px solid var(--line);
  margin-bottom: 12px;
}

.card .name,
.card .type,
.card .price,
.card .card-desc {
  padding: 0 16px;
}

.name {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 2px;
}

.type {
  color: var(--steel);
  font-size: 0.88rem;
  margin-bottom: 6px;
}

.card-desc {
  color: var(--steel);
  font-size: 0.82rem;
  line-height: 1.4;
  margin-bottom: 10px;
  flex: 1;
}

.price {
  color: var(--brass);
  font-weight: 700;
  font-size: 1.1rem;
  font-variant-numeric: tabular-nums;
}

.card-actions {
  position: absolute;
  top: 10px;
  right: 10px;
  display: flex;
  gap: 6px;
  z-index: 5;
}
```

```
.card-act-btn {
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.95);
  border: 1px solid var(--line);
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  font-size: 0.85rem;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.08);
  transition: transform 0.15s ease;
}

.card-act-btn:hover {
  transform: scale(1.08);
}

.fav-btn {
  width: 34px;
  height: 34px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.95);
  border: 1px solid var(--line);
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.08);
  transition: all 0.15s ease;
}

.fav-btn:hover {
  transform: scale(1.1);
  border-color: var(--brass);
}

.fav-btn--active {
  background: #fff;
}

/*
------------------------------------------------------------
-----------------
 * State Boxes (Loading, Error, Empty)
                                                            *
------------------------------------------------------------
----------------- */
.state-box {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  text-align: center;
```

```
  padding: 56px 24px;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  margin: 24px 0;
  gap: 12px;
}

.state-box p {
  color: var(--steel);
  margin: 0;
  max-width: 48ch;
}

.state-box--error {
  border-color: #f1948a;
  color: #c0392b;
}

.state-box--error p {
  color: #c0392b;
}

.state-box__actions {
  display: flex;
  gap: 12px;
  margin-top: 8px;
}

.local-notice {
  font-size: 0.85rem;
  color: var(--steel);
  text-align: center;
  margin: 18px 0;
}

.link-btn {
  background: none;
  border: none;
  color: var(--brass);
  font: inherit;
  font-weight: 600;
  cursor: pointer;
  text-decoration: underline;
  padding: 0;
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}

.spin {
  animation: spin 1s linear infinite;
}
```

```
/*
------------------------------------------------------------
-----------------
 * Product Detail View
                                                            *
------------------------------------------------------------
----------------- */
.detail {
  padding: 24px 0 64px;
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.detail-back {
  background: none;
  border: none;
  color: var(--steel);
  font: inherit;
  font-weight: 600;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 0;
  align-self: flex-start;
  transition: color 0.15s ease;
}

.detail-back:hover {
  color: var(--ink);
}

.detail-hero {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 28px;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  overflow: hidden;
}

@media (max-width: 680px) {
  .detail-hero {
    grid-template-columns: 1fr;
  }
}

.detail-img {
  width: 100%;
  height: 100%;
  min-height: 280px;
  max-height: 420px;
  object-fit: cover;
  background: #f7f9fa;
```

```
  border-right: 1px solid var(--line);
}

.detail-info {
  padding: 28px 24px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  justify-content: center;
}

.detail-info h1 {
  font-size: 2.2rem;
  margin: 0;
}

.detail-price {
  font-size: 1.8rem;
  margin: 8px 0 0;
}

.detail-rating {
  display: flex;
  align-items: center;
  gap: 6px;
  color: var(--ink);
  font-weight: 600;
  font-size: 0.95rem;
  margin: 4px 0 16px;
}

.detail-actions {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 16px;
}

.detail-section {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 24px;
}

.detail-section h2 {
  font-size: 1.25rem;
  margin: 0 0 16px;
  border-bottom: 1px solid var(--line);
  padding-bottom: 8px;
}

.detail-specs {
  list-style: none;
  padding: 0;
  margin: 0;
```

```
  display: grid;
    grid-template-columns:  repeat(auto-fill,  minmax(200px,
1fr));
  gap: 12px;
}

.detail-specs li {
  background: var(--paper);
  padding: 10px 14px;
  border-radius: 4px;
  font-size: 0.92rem;
}

.detail-steps {
  margin: 0;
  padding-left: 20px;
  display: flex;
  flex-direction: column;
  gap: 10px;
  color: var(--steel);
}

.detail-steps li {
  line-height: 1.5;
}

/* Reviews */
.review-form {
  display: flex;
  flex-direction: column;
  gap: 12px;
  background: var(--paper);
  padding: 18px;
  border-radius: 4px;
  margin-bottom: 24px;
}

.review-stars {
  display: flex;
  gap: 4px;
}

.star-btn {
  background: none;
  border: none;
  padding: 2px;
  cursor: pointer;
  display: flex;
  align-items: center;
}

.review-input {
  width: 100%;
  padding: 10px 12px;
  font: inherit;
  font-size: 0.92rem;
```

```
  border: 1px solid var(--line);
  border-radius: 4px;
  resize: vertical;
}

.review-input:focus {
  outline: none;
  border-color: var(--brass);
}

.review-list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.review-item {
  border-bottom: 1px solid var(--line);
  padding-bottom: 12px;
}

.review-item:last-child {
  border-bottom: none;
  padding-bottom: 0;
}

.review-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 4px;
}

.review-user {
  font-weight: 600;
  font-size: 0.92rem;
}

.review-stars-display {
  display: inline-flex;
  gap: 2px;
}

.review-comment {
  margin: 4px 0 0;
  color: var(--steel);
  font-size: 0.9rem;
}

.form-error {
  color: #c0392b;
  font-size: 0.85rem;
  margin: 0;
```

```
}

.form-success {
  color: #27ae60;
  font-size: 0.85rem;
  margin: 0;
}

.form-hint {
  color: var(--steel);
  font-size: 0.82rem;
  margin: 0;
}

/*
------------------------------------------------------------
-----------------
 * Form Pages (CreateGunPage, EditGunPage)
                                                            *
------------------------------------------------------------
----------------- */
.form-page {
  padding: 32px 0 64px;
}

.form-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 24px;
}

.form-title-group {
  margin-bottom: 28px;
}

.form-title-group h1 {
  margin: 0 0 8px;
}

.form-steps-nav {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 28px;
}

.step-badge {
  padding: 8px 16px;
  font-size: 0.85rem;
  font-weight: 600;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  color: var(--steel);
}
```

```
.step-badge--active {
  background: var(--ink);
  color: #fff;
  border-color: var(--ink);
}

.step-badge--done {
  border-color: var(--brass);
  color: var(--brass);
}

.step-badge-connector {
  flex: 1;
  height: 1px;
  background: var(--line);
}

.card-box {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 24px;
  margin-bottom: 24px;
}

.card-box-title {
  font-size: 1.25rem;
  margin: 0 0 6px;
}

.card-box-desc {
  color: var(--steel);
  font-size: 0.9rem;
  margin: 0 0 18px;
}

.upload-dropzone {
  border: 2px dashed var(--line);
  border-radius: 6px;
  padding: 36px 20px;
  text-align: center;
  background: var(--paper);
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
}

.hidden-file-input {
  display: none;
}

.upload-label {
  cursor: pointer;
  display: flex;
```

```
  flex-direction: column;
  align-items: center;
  gap: 8px;
}

.upload-icon {
  color: var(--steel);
}

.upload-main-text {
  font-weight: 600;
  font-size: 1.05rem;
  color: var(--ink);
}

.upload-sub-text {
  font-size: 0.85rem;
  color: var(--steel);
}

.preset-selector {
  margin-top: 14px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
}

.preset-title {
  font-size: 0.85rem;
  color: var(--steel);
}

.preset-pills {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}

.preview-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
}

.preview-image-wrap {
  position: relative;
  width: 100%;
  max-width: 420px;
  height: 260px;
  border: 1px solid var(--line);
  border-radius: 4px;
  overflow: hidden;
}
```

```
.preview-img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.preview-remove-btn {
  position: absolute;
  top: 10px;
  right: 10px;
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: rgba(0, 0, 0, 0.7);
  color: #fff;
  border: none;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
}

.preview-actions {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
  justify-content: center;
}

.selected-thumb-bar {
  display: flex;
  align-items: center;
  gap: 14px;
  padding: 12px 16px;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  margin-bottom: 20px;
}

.thumb-small {
  width: 50px;
  height: 50px;
  object-fit: cover;
  border-radius: 4px;
  border: 1px solid var(--line);
}

.thumb-info {
  flex: 1;
  display: flex;
  flex-direction: column;
}

.thumb-title {
  font-weight: 600;
```

```
  font-size: 0.95rem;
}

.thumb-sub {
  font-size: 0.8rem;
  color: var(--steel);
}

.entry-form {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.form-grid--3 {
  grid-template-columns: 1fr 1fr 1fr;
}

@media (max-width: 600px) {
  .form-grid,
  .form-grid--3 {
    grid-template-columns: 1fr;
  }
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.form-label {
  font-size: 0.82rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--steel);
}

.req {
  color: #c0392b;
}

.form-input,
.form-select,
.form-textarea {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid var(--line);
  border-radius: 4px;
```

```
  background: #fff;
  font: inherit;
  font-size: 0.95rem;
  color: var(--ink);
}

.form-input:focus,
.form-select:focus,
.form-textarea:focus {
  outline: none;
  border-color: var(--brass);
}

.form-fieldset {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 18px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.fieldset-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.multi-rows {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.multi-row {
  display: grid;
  grid-template-columns: 1fr 1fr auto;
  gap: 10px;
  align-items: center;
}

.step-row {
  display: flex;
  align-items: center;
  gap: 10px;
}

.step-num {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  background: var(--paper);
  border: 1px solid var(--line);
  display: flex;
  align-items: center;
```

```
  justify-content: center;
  font-weight: 700;
  font-size: 0.82rem;
  flex-shrink: 0;
}

.checkbox-row {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 16px;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
}

.form-checkbox {
  width: 18px;
  height: 18px;
  accent-color: var(--brass);
  cursor: pointer;
}

.checkbox-label {
  font-size: 0.92rem;
  color: var(--ink);
  cursor: pointer;
}

.form-actions-bar {
  display: flex;
  justify-content: flex-end;
  gap: 12px;
  padding-top: 16px;
  border-top: 1px solid var(--line);
}

.edit-img-wrap {
  display: flex;
  align-items: center;
  gap: 20px;
}

.edit-current-img {
  width: 140px;
  height: 100px;
  object-fit: cover;
  border-radius: 4px;
  border: 1px solid var(--line);
}

.edit-img-controls {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
```

```
/*
------------------------------------------------------------
-----------------
 * Profile Page
                                                            *
------------------------------------------------------------
----------------- */
.profile-page {
  padding: 32px 0 64px;
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.profile-card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 24px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.profile-card-top {
  display: flex;
  align-items: center;
  gap: 20px;
}

.avatar-wrap {
  position: relative;
  width: 80px;
  height: 80px;
  border-radius: 50%;
  overflow: visible;
}

.avatar-img {
  width: 100%;
  height: 100%;
  border-radius: 50%;
  object-fit: cover;
  border: 2px solid var(--line);
}

.avatar-placeholder {
  width: 100%;
  height: 100%;
  border-radius: 50%;
  background: var(--paper);
  border: 2px solid var(--line);
  display: flex;
  align-items: center;
  justify-content: center;
```

```
}

.avatar-upload-btn {
  position: absolute;
  bottom: 0;
  right: 0;
  width: 26px;
  height: 26px;
  border-radius: 50%;
  background: var(--ink);
  color: #fff;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  box-shadow: 0 2px 6px rgba(0,0,0,0.2);
}

.profile-identity h2 {
  margin: 0 0 6px;
  font-size: 1.8rem;
}

.profile-id-row {
  display: flex;
  align-items: center;
  gap: 8px;
}

.mono-badge {
     font-family:  ui-monospace,  SFMono-Regular,  Menlo,
Monaco, Consolas, monospace;
  font-size: 0.82rem;
  background: var(--paper);
  padding: 4px 8px;
  border: 1px solid var(--line);
  border-radius: 4px;
  color: var(--steel);
}

.profile-stats {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
  border-top: 1px solid var(--line);
  padding-top: 18px;
}

.stat-item {
  display: flex;
  flex-direction: column;
  gap: 4px;
}

.stat-val {
  font-size: 1.8rem;
```

```
  font-weight: 700;
  font-family: 'Arial Narrow', sans-serif;
  color: var(--ink);
}

.stat-label {
  font-size: 0.82rem;
  color: var(--steel);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.profile-form {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 24px;
  display: flex;
  flex-direction: column;
  gap: 18px;
}

.section-title {
  margin: 0;
  font-size: 1.15rem;
  font-weight: 700;
}

.profile-danger-zone {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 24px;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.danger-actions {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
}

/*
-----------------------------------------------------------
-----------------
 * Modal Dialog (ConfirmModal)
                                                           *
-----------------------------------------------------------
----------------- */
.modal-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(24, 34, 44, 0.65);
  backdrop-filter: blur(4px);
```

```
  z-index: 100;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
}

.modal-box {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 6px;
  width: 100%;
  max-width: 440px;
  padding: 28px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  box-shadow: 0 12px 32px rgba(0, 0, 0, 0.18);
  animation: modalIn 0.2s ease-out;
}

@keyframes modalIn {
  from { opacity: 0; transform: scale(0.95); }
  to { opacity: 1; transform: scale(1); }
}

.modal-icon {
  margin-bottom: 4px;
}

.modal-title {
  font-size: 1.5rem;
  margin: 0;
}

.modal-msg {
  color: var(--steel);
  font-size: 0.95rem;
  line-height: 1.5;
  margin: 0 0 12px;
}

.modal-actions {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
}

.modal-btn {
  padding: 9px 18px;
  font: inherit;
  font-size: 0.92rem;
  font-weight: 600;
  border-radius: 4px;
  cursor: pointer;
  border: 1px solid transparent;
```

```
}

.modal-btn--cancel {
  background: #fff;
  color: var(--ink);
  border-color: var(--line);
}

.modal-btn--confirm {
  background: var(--ink);
  color: #fff;
}

.modal-btn--danger {
  background: #c0392b;
  color: #fff;
}

/*
------------------------------------------------------------
-----------------
 * Toast Notifications
                                                            *
------------------------------------------------------------
----------------- */
.toast {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 18px;
  border-radius: 4px;
  font-size: 0.92rem;
  font-weight: 500;
  margin-bottom: 20px;
}

.toast--error {
  background: #feeceb;
  color: #c0392b;
  border: 1px solid #f5b7b1;
}

.toast--success {
  background: #e8f8f5;
  color: #1e8449;
  border: 1px solid #a3e4d7;
}

/*
------------------------------------------------------------
-----------------
 * Footer
                                                            *
------------------------------------------------------------
----------------- */
.footer {
```

```
  margin-top: auto;
  padding: 24px;
  text-align: center;
  background: var(--panel);
  color: var(--panel-text);
  font-size: 0.9rem;
}

.footer p {
  margin: 0;
}

/*
------------------------------------------------------------
-----------------
 * PWA Badge & Installation Prompt
                                                            *
------------------------------------------------------------
----------------- */
.pwa-badge {
  position: fixed;
  bottom: 24px;
  right: 24px;
  background: #fff;
  border: 1px solid var(--line);
  box-shadow: 0 8px 24px rgba(24, 34, 44, 0.14);
  padding: 12px 16px;
  border-radius: 6px;
  z-index: 50;
  display: flex;
  align-items: center;
  gap: 14px;
  max-width: 420px;
  animation: slideUp 0.3s ease-out;
}

.pwa-badge--offline {
  background: var(--paper);
  border-color: var(--brass);
  color: var(--ink);
  font-size: 0.88rem;
  font-weight: 500;
}

.pwa-badge-content {
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 0.9rem;
}

.pwa-badge-actions {
  display: flex;
  align-items: center;
  gap: 8px;
}
```

```
@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

8. Tambahkan helper di dalam folder src

src/utils/draftStorage.js

```
//
----------------------------------------------------------
-----------------
// draftStorage.js — auto-save / restore form drafts via
localStorage.
//
// Drafts are keyed by a string id:
//   'create'              → the Create form
//   '<uuid>'              → the Edit form for a specific
product
//
// Each draft is stored under  `gun_draft_<id>`  and its
companion
// timestamp under  `gun_draft_<id>_ts`.
//
----------------------------------------------------------
-----------------

const PREFIX = 'gun_draft_'
const TS_SUFFIX = '_ts'

/**
 * Persist a draft object.
 * @param {Object} draftData — arbitrary JSON-serializable
form state
 * @param {string} draftId   — 'create' or a product UUID
 * @returns {boolean} true on success
 */
export const saveDraft = (draftData, draftId = 'create')
=> {
  try {
    const key = `PREFIX{draftId}`
    const ts = new Date().toISOString()
    localStorage.setItem(key, JSON.stringify({
...draftData, savedAt: ts }))
    localStorage.setItem(`key{TS_SUFFIX}`, ts)
    return true
```

```
  } catch {
    return false
  }
}

/**
 * Load a previously saved draft.
 * @returns {Object|null}
 */
export const loadDraft = (draftId = 'create') => {
  try {
    const raw = localStorage.getItem(`PREFIX{draftId}`)
    return raw ? JSON.parse(raw) : null
  } catch {
    return null
  }
}

/**
 * Remove a draft and its timestamp from storage.
 */
export const deleteDraft = (draftId = 'create') => {
  try {
    const key = `PREFIX{draftId}`
    localStorage.removeItem(key)
    localStorage.removeItem(`key{TS_SUFFIX}`)
    return true
  } catch {
    return false
  }
}

/**
 * Check whether a draft exists.
 */
export const hasDraft = (draftId = 'create') => {
  try {
    return localStorage.getItem(`PREFIX{draftId}`) !==
null
  } catch {
    return false
  }
}

/**
 * Return the ISO-8601 timestamp string of the last save,
or null.
 */
export const getDraftTimestamp = (draftId = 'create') => {
  try {
    return localStorage.getItem(`PREFIX
{draftId}${TS_SUFFIX}`)
  } catch {
    return null
  }
```

```
}

/**
 * Human-readable "time ago" label for a draft timestamp.
 */
export const formatDraftTime = (timestamp) => {
  if (!timestamp) return ''
  const diff = Date.now() - new Date(timestamp).getTime()
  const mins = Math.floor(diff / 60000)
  if (mins < 1) return 'Just now'
  if (mins < 60) return `${mins} min ago`
  const hrs = Math.floor(mins / 60)
  if (hrs < 24) return `${hrs} hr ago`
  const days = Math.floor(hrs / 24)
  if (days < 7) return `daysday{days > 1 ? 's' : ''} ago`
  return new Date(timestamp).toLocaleDateString()
}

/**
 * Return all existing draft ids.
 */
export const getAllDraftIds = () => {
  try {
    return Object.keys(localStorage)
      .filter((k) => k.startsWith(PREFIX) &&
!k.endsWith(TS_SUFFIX))
      .map((k) => k.slice(PREFIX.length))
  } catch {
    return []
  }
}

/**
 * Remove every gun draft from storage.
 */
export const clearAllDrafts = () => {
  getAllDraftIds().forEach(deleteDraft)
}
```

src/utils/helpers.js

```
//
-------------------------------------------------------------
-----------------
// helpers.js — pure utility functions used across the
application.
//
-------------------------------------------------------------
-----------------

/**
 * Format a numeric price as "$1,234".
```

```
 */
export const formatPrice = (price) => {
  const num = Number(price)
  if (Number.isNaN(num)) return '$0'
  return `$${num.toLocaleString()}`
}

/**
 * Client-side text search — case-insensitive match
against name and
 * description.
 */
export const matchesSearch = (item, query) => {
  if (!query) return true
  const q = query.toLowerCase()
  return (
    (item.name || '').toLowerCase().includes(q) ||
    (item.description || '').toLowerCase().includes(q)
  )
}

/**
 * Client-side category / type filter.
 * `category` of '' or 'All' matches everything.
 */
export const matchesCategory = (item, category) => {
  if (!category || category === 'All') return true
  // Support both the local `type` field and the backend
`category` field
  return (
    (item.type || '').toLowerCase() ===
category.toLowerCase() ||
    (item.category || '').toLowerCase() ===
category.toLowerCase()
  )
}

/**
 * Sort an array of items by a given key.
 * @param {'name'|'price'} key
 * @param {'asc'|'desc'}   order
 */
export const sortItems = (items, key = 'name', order =
'asc') => {
  const sorted = [...items].sort((a, b) => {
    if (key === 'price') return (Number(a.price) || 0) -
(Number(b.price) || 0)
    return String(a[key] ||
'').localeCompare(String(b[key] || ''))
  })
  return order === 'desc' ? sorted.reverse() : sorted
}

/**
 * Truncate a string to `max` characters, adding "…" when
```

```
clipped.
 */
export const truncate = (str, max = 100) => {
  if (!str || str.length <= max) return str || ''
  return str.slice(0, max).trimEnd() + '…'
}

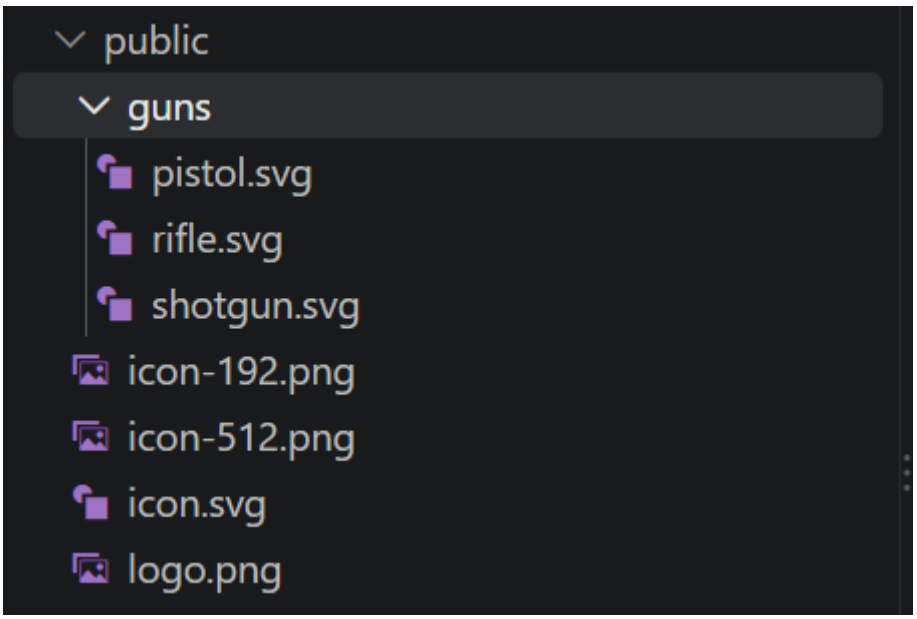
/**
 * Extract unique category values from a list of items.
 * Returns ['All', ...categories].
 */
export const extractCategories = (items) => {
  const set = new Set()
  items.forEach((i) => {
    const cat = i.type || i.category
    if (cat) set.add(cat)
  })
  return ['All', ...Array.from(set).sort()]
}

/**
 * Compute average rating from an array of review objects
that each have a
 * numeric `rating` property.  Returns 0 when empty.
 */
export const averageRating = (reviews) => {
  if (!reviews || reviews.length === 0) return 0
  const sum = reviews.reduce((acc, r) => acc +
(Number(r.rating) || 0), 0)
  return Math.round((sum / reviews.length) * 10) / 10
}
```

9. Atur *icon*/logo untuk PWA

   Vite.config.js

   Link *download* Icons



Gambar 6. 3 *file icon* PWA

10. *Install* semua *dependencies* yang dibutuhkan dan cek PWA kalian

```
npm install lucide-react
npm install -D vite-plugin-pwa
npm install tailwindcss @tailwindcss/vite
npm install axios
```

Mode pengembangan

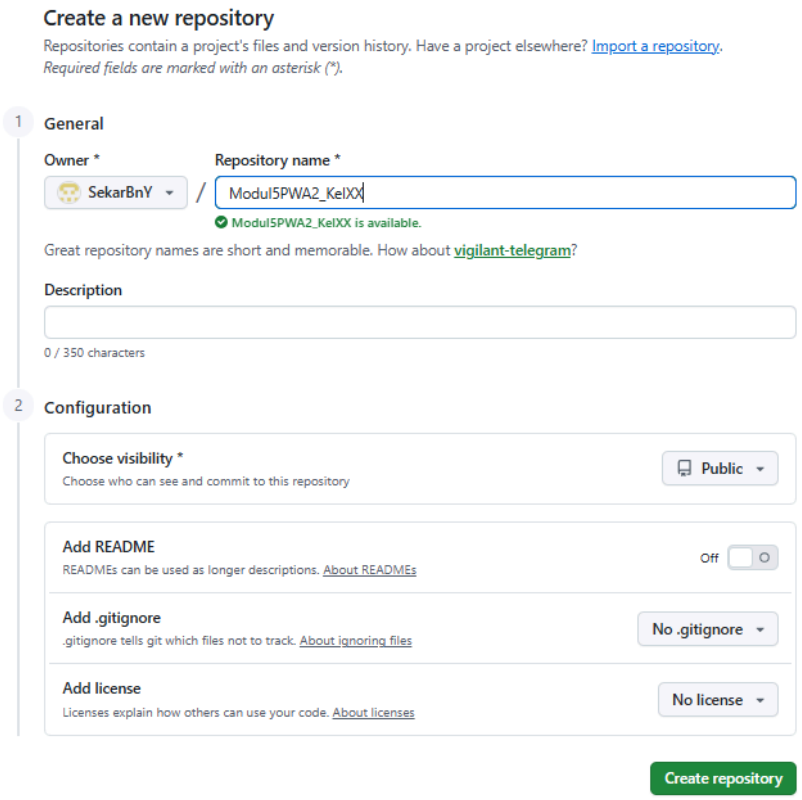
```
npm run dev
```

Mode Produksi

```
npm run build
```

6.4.3. *Deploy* PWA ke Vercel

1. Buat *repository* baru pada Github



Gambar 6. 4 Repository baru untuk modul 5

2. *Copy link repository* yang telah dibuat dan masukkan *command* berikut

```
>  git remote add origin [link repo]
```

3. *Staged* perubahan kode yang ada pada *project* dengan *command* berikut

```
>  git add .
```

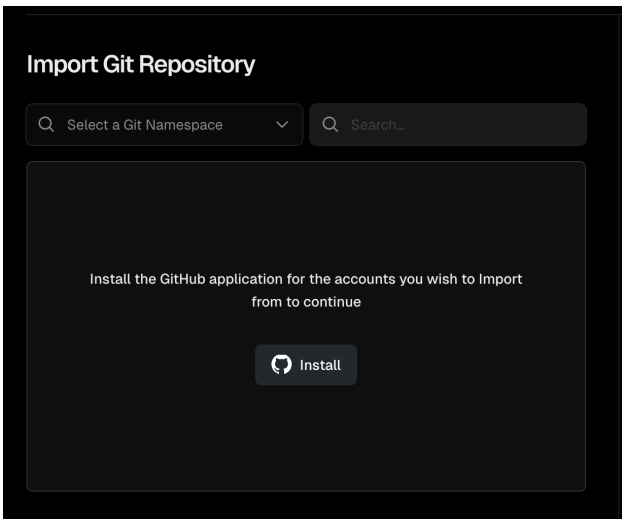
4. *Commit* perubahan dengan *command* berikut

```
>  git commit -m "[isi dengan pesan kalian]"
```

5. *Push project* ke *repository* Github dengan *command* berikut

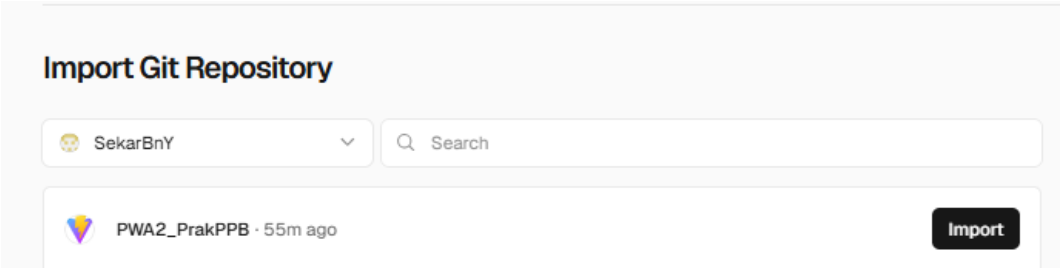
```
>  git push -u origin main
```

6. Buka Vercel dan *login* menggunakan Github kemudian *install* Github *app* untuk mengakses *repository* kalian



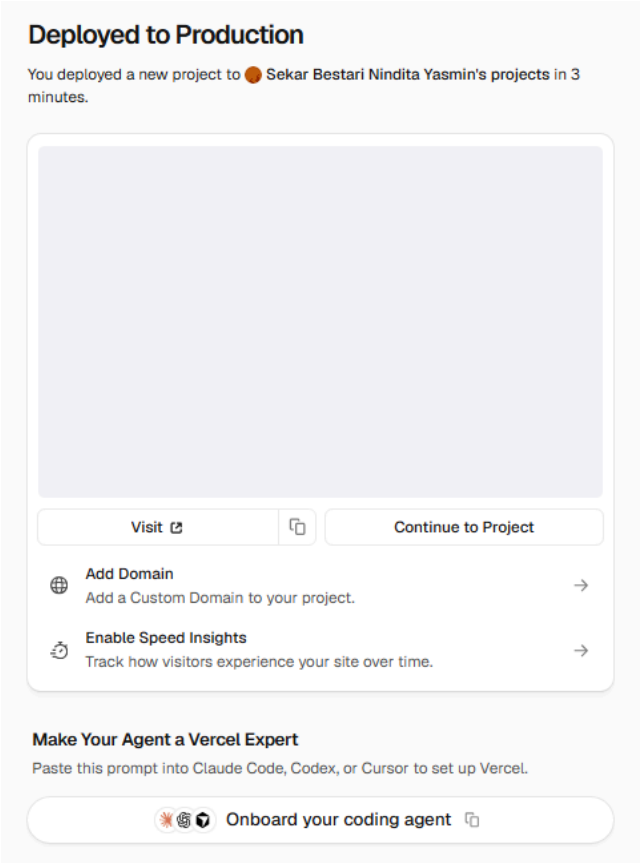
Gambar 6. 5 Import GitHub Repository

7. Pilih *repository* Github



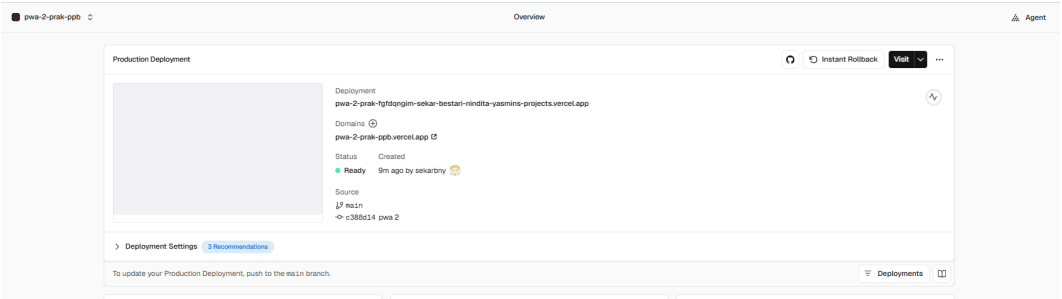
Gambar 6. 6 Pilih *repository* yang ingin di-*deploy*

8. *Import repository* percobaan dan PWA akan otomatis ter-*deploy*



Gambar 6. 7 PWA berhasil diunggah dan dapat diakses

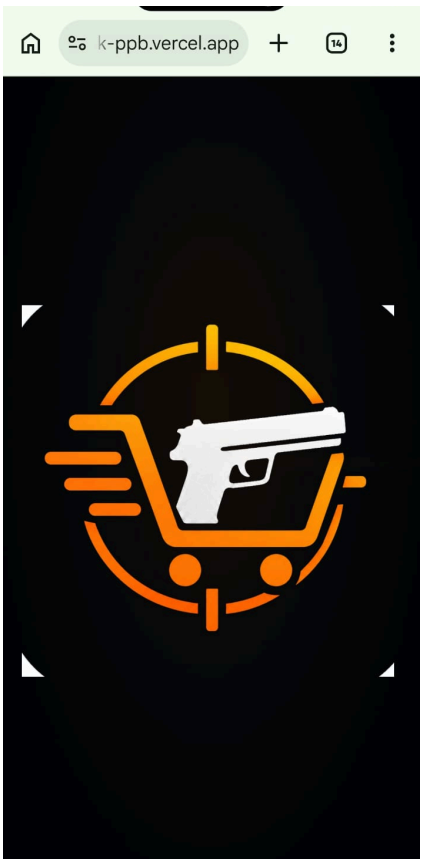
9. PWA bisa dicek di *domain* yang diberikan Vercel



Gambar 6. 8 Tampilan PWA pada Vercel

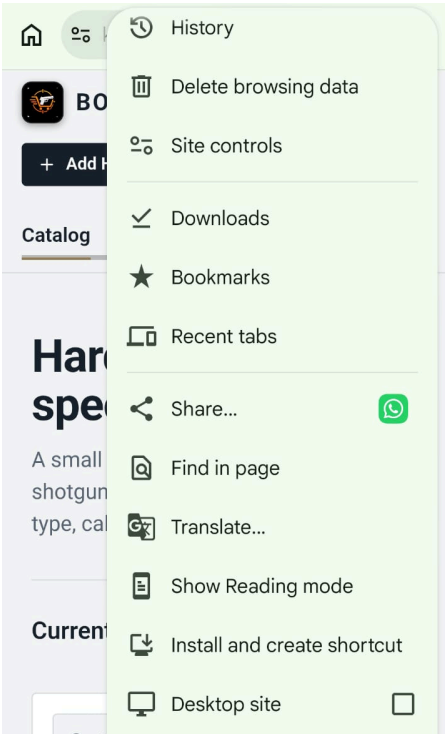
6.4.4. PWA pada Mobile

1. Buka *link* PWA yang telah di-*deploy*



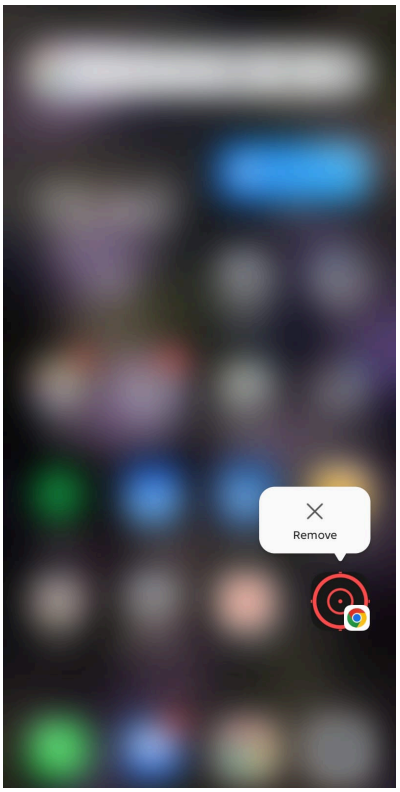
Gambar 6. 9 Buka *link* PWA

2. Klik *icon share* yang terletak di samping *link*, lalu *scroll* dan pilih “Add to Home Screen” atau “Install and Create Shortcut”



Gambar 6. 10 Add to Home Screen

3. PWA akan tampil pada *smartphone* kalian



Gambar 6. 11 Tampilan PWA pada *smartphone*